Codebook of UltraGrammer

Contents

1 解題流程 **1**
- 1.1 本 codebook 的用法 . . . 1
- 1.2 看 N 決定複雜度 (1 秒約 10^8 次運算) . . . 1
- 1.3 題目關鍵字 → 先想這些 . . . 2
- 1.4 模板不能直接套時 (題目變形) . . . 2
- 1.5 嘗試... . . . 2
- 1.6 檢查... . . . 2

2 基礎 **2**
- 2.1 模板 . . . 2
- 2.2 Fast IO 與 __int128 . . . 2
- 2.3 C++ 函式庫速查 . . . 2
- 2.4 位元與數學小技巧 . . . 3

3 資料結構 **3**
- 3.1 BIT (Fenwick Tree) . . . 3
- 3.2 Segment Tree 線段樹 (lazy) . . . 3
- 3.3 Sparse Table 稀疏表 . . . 3
- 3.4 Prefix Sum 前綴和與 Difference 差分 . . . 3
- 3.5 Monotonic Stack 單調棧 . . . 3
- 3.6 Monotonic Queue 單調隊列 . . . 4
- 3.7 Disjoint Set 並查集 . . . 4
- 3.8 座標壓縮、pb_ds 平衡樹 . . . 4

4 數學 **4**
- 4.1 快速冪 . . . 4
- 4.2 模逆元 (費馬小定理) . . . 4
- 4.3 擴展歐幾里得 . . . 4
- 4.4 組合數 C(n,k) . . . 4
- 4.5 質數、質因數分解、線性篩 . . . 5
- 4.6 尤拉函數 φ . . . 5
- 4.7 Legendre's Formula 勒讓德公式 . . . 5
- 4.8 數論分塊 . . . 5
- 4.9 排容原理 . . . 5
- 4.10 矩陣快速冪 . . . 5
- 4.11 高斯消去、XOR 線性基 . . . 5
- 4.12 大數運算 . . . 6
- 4.13 博弈論 (Nim、SG 函數) . . . 6
- 4.14 常用公式 . . . 6

5 字串 **6**
- 5.1 KMP 字串匹配 . . . 6
- 5.2 Z-Algorithm . . . 6
- 5.3 Manacher 迴文 . . . 6
- 5.4 Trie 字典樹 . . . 7
- 5.5 AC 自動機 . . . 7

6 圖論 **7**
- 6.1 BFS、拓樸排序、網格 BFS、0-1 BFS . . . 7
- 6.2 DFS . . . 7
- 6.3 Dijkstra 單源最短路 . . . 8
- 6.4 分層圖 / 狀態圖 (最短路 + 額外條件) . . . 8
- 6.5 SPFA (Bellman-Ford) . . . 8
- 6.6 Floyd-Warshall 全點對最短路 . . . 8
- 6.7 二分圖判定 (著色) . . . 8
- 6.8 割點、橋、點雙連通分量 . . . 9
- 6.9 匈牙利演算法 (二分圖最大匹配) . . . 9
- 6.10 KM 演算法 . . . 9
- 6.11 Dinic 最大流 . . . 9

7 樹 **10**
- 7.1 Diameter 直徑 . . . 10
- 7.2 Traversal 遍歷 . . . 10
- 7.3 LCA 最近共同祖先 (倍增) . . . 10
- 7.4 Kruskal 最小生成樹 . . . 10
- 7.5 嚴格次小生成樹 . . . 10

8 幾何 **11**
- 8.1 點與向量基礎 . . . 11
- 8.2 線段相交、直線交點 . . . 11
- 8.3 點與直線/線段 . . . 11
- 8.4 多邊形面積、點在多邊形內 . . . 11
- 8.5 凸包 Convex Hull、旋轉卡尺 . . . 11
- 8.6 圓 . . . 12

9 DP **12**
- 9.1 背包問題 . . . 12
- 9.2 分錢幣問題 . . . 12
- 9.3 LCS 最長共同子序列、編輯距離 . . . 12
- 9.4 LIS 最長遞增子序列 . . . 13
- 9.5 LCS 轉 LIS . . . 13
- 9.6 區間 DP . . . 13
- 9.7 加權區間排程 (工作有獎勵) . . . 13
- 9.8 Kadane 最大子陣列和 . . . 13
- 9.9 跳格子 (路徑還原) . . . 14

10 Greedy **14**
- 10.1 區間覆蓋 (灑水器) . . . 14
- 10.2 區間選點 . . . 14
- 10.3 最多不重疊區間 . . . 14
- 10.4 最少會議室 . . . 14
- 10.5 掃描線：最大重疊數 . . . 14
- 10.6 最小化最大延遲 . . . 14
- 10.7 有期限的單位工作排程 . . . 14
- 10.8 多機排程 (LPT 近似) . . . 15
- 10.9 跳躍遊戲 (最少跳幾次) . . . 15
- 10.10Josephus 約瑟夫問題 . . . 15
- 10.11Huffman 合併與編碼 . . . 15

11 搜尋與技巧 **15**
- 11.1 Binary Search 二分搜 . . . 15
- 11.2 Ternary Search 三分搜 . . . 15
- 11.3 Two Pointer 雙指針 . . . 16
- 11.4 Sliding Window 滑動視窗 . . . 16
- 11.5 暴力枚舉、回溯、折半枚舉 . . . 16

12 MISC **16**
- 12.1 安全的 Hash . . . 16
- 12.2 對拍器 (測資產生器) . . . 16

1 解題流程

1.1 本 codebook 的用法

- 每份 code 都假設已經有「模板」的 `bits/stdc++.h`、`using namespace std`、`typedef long long ll`。全部是**全域陣列 + 函式**，沒有 struct。
- 註解 `// [改]` 的那一行 = 題目變形時「通常要改的地方」。先找 [改]，再看檔案最後的 [變形] 清單。
- 每份檔案各自獨立。同時貼兩份時，重複的 `MAXN / INF / n / g` 要刪掉或改名。需要別的檔案時註解會寫「需要 XXX.cpp」。
- 多筆測資：全域陣列、`g[i].clear()`、計數器，每筆都要重設。

1.2 看 N 決定複雜度 (1 秒約 10^8 次運算)

N 的範圍	複雜度	可能的作法
≤ 10	$O(N!)$	全排列、回溯
≤ 20	$O(2^N N)$	位元枚舉、選/不選遞迴
≤ 40	$O(2^{N/2})$	折半枚舉
≤ 500	$O(N^3)$	Floyd、區間 DP、KM
≤ 5000	$O(N^2)$	二維 DP、LCS、枚舉兩端點
$\le 2 \cdot 10^5$	$O(N \log N)$	排序、二分、BIT/線段樹、Dijkstra
$\le 10^6$	$O(N)$	雙指針、前綴和、KMP/Z、線性篩
$\le 10^{12}$	$O(\sqrt{N})$	因數分解、數論分塊
$\le 10^{18}$	$O(\log N)$	快速冪、矩陣快速冪、二分答案

1.3 題目關鍵字 → 先想這些

看到…	想到…
最大值最小、最小值最大、至少需要多少	對答案二分搜 + greedy 檢查
最少步數 (每步代價相同)	BFS (網格、多源、0-1 BFS)
最短路 (有權 / 負權 / 全點對)	Dijkstra / SPFA / Floyd
最短路 + 多一個條件 (次數、鑰匙、時間、油量)	**分層圖 / 狀態圖**
先後順序、依賴關係、有沒有環	拓樸排序、DFS 判環
合併、是否同一組、連通塊	DSU、DFS
花最少成本把所有點連起來	MST (Kruskal)
區間和、大量區間修改	前綴和、差分、BIT、線段樹
區間最大最小 (不修改)、視窗最大	Sparse Table、單調隊列
左/右邊第一個更大 (更小)	單調棧
子陣列和 $= k$	非負: 雙指針；有負: 前綴和 + map
找字串 / 多個字串 / 迴文	KMP、Z / AC 自動機 / Manacher
方法數 mod p、$C(n,k)$、除法取模	組合數、模逆元
遞迴式但 n 到 10^{18}	矩陣快速冪
選或不選、容量限制	背包 DP
配對、分派 (有權重 / 有容量)	匈牙利 (KM / Dinic)
拔掉後會斷開的點/邊	割點、橋 (BCC)
樹上兩點路徑、距離	LCA (倍增)
兩人輪流、誰必勝	Nim、SG 函數、打表找規律
排程、選最多不重疊區間	Greedy (依結束時間排序)
不重疊的工作、每個有獎勵，求最大獎勵	加權區間排程 DP
$N \le 20$、分成兩組、每個選或不選	選/不選遞迴 $O(2^N)$ (Brute.cpp 5.)，不是排列
相鄰只差一個 bit 的序列	Gray code：$i \oplus (i \gg 1)$
重新排列成迴文	奇數次的字母 ≤ 1；左半 + 中間 + 反轉左半
所有相異排列 (字母會重複)	`cnt[26]` 回溯、每層試 a∼z (Brute.cpp 6.)；或 sort + next_permutation

1.4 模板不能直接套時 (題目變形)

1. **多了一個條件** → 把它塞進「狀態」：最短路變成 (點, 條件) 的分層圖；DP 多開一維。狀態數 × 轉移數 $\le 10^8$ 就能做。
2. **終點固定、起點很多** → 反向建圖從終點跑一次；**起點很多** → 超級源點或多源 BFS。
3. **直接求很難、但「猜答案 x」後很好檢查** → 二分搜答案。
4. **換個問法**：總數 − 不合法數；最小割 = 最大流；最小點覆蓋 = 最大匹配；最大獨立集 = $N-$ 匹配。
5. **換枚舉的東西**：枚舉答案、枚舉中間點、枚舉比較小的那一維；「所有子陣列的總和」→ 算每個元素被算幾次 (貢獻法)。
6. **離線處理**：把詢問排序；刪除很難 → 倒過來變加入 (DSU)。
7. **化簡**：先排序 (很多東西會變單調)；$|x_1 - x_2| + |y_1 - y_2|$ 轉成 $(x+y, x-y)$ 取 max；值域大但個數少 → 座標壓縮。
8. **完全沒想法**：寫暴力把小範圍答案印出來找規律 (打表)，再用對拍驗證。

1.5 嘗試...

- **再讀一遍題目**，用範例手動模擬一次
- 懷疑你認為「理所當然」的直覺？試著寫下直覺的規律 (有助於觀察)
- 換個方向思考？暴力解？貪心解？對答案二分搜？動態規劃？
- 狀態建模，把問題轉化成圖論？
- 只在奇數 index 反轉狀態？用乘法取代除法？($/2 = \times 2^{-1}$，模逆元)

1.6 檢查...

- **確定沒漏看敘述？** 題目額外的敘述、暗示？輸出格式 (空白、換行、大小寫)？
- 數值範圍：溢位 (`int*int` 先轉 `1LL*`)、陣列大小、精度位數
- 多筆測資之間的初始化？$N=0$、$N=1$、全部相同、找不到答案時的輸出？
- 圖：不連通？自環、重邊？鏈狀樹 (遞迴太深)？$V=1, E=0$？
- 模板抄錯？不同陣列/字串用同一個 index 而越界？非 void 函式忘記 return？
- **複雜度看起來對卻 TLE？** 迴圈裡 `s = c + s`、`s = s + c` 每次 $O(N)$ (改用 `+=`)；組合題的 DFS 寫成排列 ($N!$)。
- 回溯時共用的狀態 (`visited`、`cnt`、字串) **每一個**都有還原嗎？位元運算 `&` `^` `|` 優先順序比 `==` 低，要加括號。
- 浮點數誤差 (用 eps，必要時 long double)？負數取模 `(x%m+m)%m`？除以 0？
- signed 與 unsigned 比較 (`v.size()-1` 在 size 為 0 時)？
- 區間的端點：開區間還是閉區間？同一時間點的事件誰先處理？

2 基礎

2.1 模板

用法: 競賽用基礎模板。所有其他 code 都假設有這份的前 6 行。

```cpp
#include <bits/stdc++.h>
#pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2")  // 評測機 CPU 不支援會 RE，不確定就不要開
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
// 各模板自己會宣告 INF / MOD / MAXN；int 的 INF 用 0x3f3f3f3f (可 memset、相加不溢位)

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr); // 之後不能混用 scanf/printf
    int T = 1;
    // cin >> T;
    while (T--) {
        // 每筆測資: 全域陣列/vector/map 要清空！
    }
    // 讀到 EOF: while (cin >> n) { ... }
    // 整行讀取: getline(cin, s)；前面有 cin >> 時先 cin.ignore()
    // 浮點輸出: cout << fixed << setprecision(6) << x;
    return 0;
}
```

2.2 Fast IO 與 ___int128

用法: 輸入超過 10^6 個數時用 `read()`；___int128 可存到約 1.7×10^{38}，但要自己印。

```cpp
// 輸入量超大 (> 1e6 個數) 才需要；一般 ios::sync_with_stdio(false) 就夠
inline ll read() {
    ll x = 0; int c = getchar(); bool neg = false;
    while (c != '-' && (c < '0' || c > '9')) c = getchar();
    if (c == '-') neg = true, c = getchar();
    while (c >= '0' && c <= '9') x = x * 10 + (c - '0'), c = getchar();
    return neg ? -x : x;
}
// __int128 (約 ±1.7e38) 不能直接 cin/cout
void print128(__int128 x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) print128(x / 10);
    putchar('0' + x % 10);
}
__int128 read128(const string &s) {
    __int128 x = 0; int i = (s[0] == '-');
    for (; i < (int)s.size(); i++) x = x * 10 + (s[i] - '0');
    return s[0] == '-' ? -x : x;
}
```

2.3 C++ 函式庫速查

用法: 常用 STL 與注意事項。

```cpp
// <algorithm>
sort(a, a + n); sort(v.begin(), v.end(), greater<int>());  // 自訂: [](auto &x, auto &y){ return ...; }
stable_sort(...);                                          // 相等元素保持原順序
reverse(b, e); unique(b, e);  // unique 前要先 sort，回傳新結尾: v.erase(unique(...), v.end())
lower_bound(b, e, x);  // 第一個 >= x；upper_bound: 第一個 > x (都要已排序)
*min_element(b, e); *max_element(b, e); accumulate(b, e, 0LL); // 注意 0LL 才是 long long
next_permutation(b, e); // 下一個字典序排列，沒有了回傳 false
nth_element(b, b + k, e); // O(N) 把第 k 小放到位置 k
iota(b, e, 0); fill(b, e, x); count(b, e, x); find(b, e, x);
__gcd(a, b); gcd(a, b); lcm(a, b);  // C++17 <numeric>
// <string>
s.substr(pos, len); s.find(t) (找不到 == string::npos); stoi(s); stoll(s); to_string(x);
// stringstream ss(line); while (ss >> word) ——行內數量不固定時
// string(k, c) = k 個 c (Python 的 c * k)；s += c / s += t 均攤 O(1)；reverse(s.begin(), s.end())
// TLE: s = c + s (插在前面) 和 s = s + c 每次都複製整個字串 O(|s|)，放迴圈裡 = O(N^2)
//   → 只往後 +=，最後再 reverse；或先開 string t(n, ' ')，兩個指標從兩端往中間填
// 迴文重排: 奇數次的字母 <= 1 個 → 左半 + (中間那個) + 反轉左半 (CSES 1755)
// 尾端: s.push_back(c) / s += c 加一個；s.pop_back() 拿掉最後一個 (空字串不可)；s.back() 看最後一個
// s.append(c) 編譯錯誤 → s.append(1, c) 或 s.append(t)；s.erase(pos) 刪 pos 到結尾，s.erase(pos, len)
// 'a' + i 是 int！ s + ('a' + i) 編譯錯誤 → s + char('a' + i)；cout << 'a' + i 會印出數字
// for (const auto &x : v) 不複製；for (string x : v) 每一圈都複製一次
// 容器
// set/map: insert, erase(x), count, find, lower_bound(x), upper_bound(x)，全部 O(log N)
// multiset 只刪一個: ms.erase(ms.find(x))；ms.erase(x) 會刪掉全部的 x
// map 走訪: for (auto &[k, v] : mp)；prev(s.end()) = 最大元素；*s.begin() = 最小
// priority_queue: 預設 max-heap；min-heap: priority_queue<T, vector<T>, greater<T>>
// deque: push_front/back, pop_front/back, 也能 [i]
// bitset<N> b: b.count(), b.set(i), b[i], b << k, b | c ——布林 DP 加速 64 倍
```

```
// vector<vector<int>> v(n, vector<int>(m, 0)); v.assign(n, x); v.resize(n);
// tuple: auto [a, b, c] = t; tie(a, b) = p;
```

2.4 位元與數學小技巧

用法: 含「只差一個 bit」判斷、運算子優先順序、Gray code。

```
/* 位元技巧
   x & -x: 最低位的 1；x & (x-1): 去掉最低位的 1；__builtin_popcountll(x): 1 的個數
   __builtin_ctzll(x): 結尾 0 的個數；63 - __builtin_clzll(x) (= __lg(x)): 最高位的位置
   a ^ b ^ b = a；a + b = (a ^ b) + 2 * (a & b)
   只差一個 bit: x = a ^ b; x && !(x & (x - 1)) 或 __builtin_popcount(a ^ b) == 1 (「XOR 是偶數」不對)
   優先順序: & ^ | 比 == 還低！ a ^ b == 0 會變成 a ^ (b == 0)；% 比 ^ 高。位元運算一律加括號
   Gray code (2^n 個、相鄰只差 1 bit): 第 i 個 = i ^ (i >> 1)；印 n 位: bitset<16>(g).to_string().substr(16 - n)
     建構: G(n) = 「0 + G(n-1)」接「1 + G(n-1) 整串順序倒過來」(是倒序，不是每個 bit 翻轉)
   數學小技巧
   ceil(a / b) (a,b > 0) = (a + b - 1) / b；負數取模: ((a % m) + m) % m
   排序比較 a/b < c/d → a*d < c*b (b,d > 0，注意溢位)
   long double 精度約 18 位；sqrt 整數: ll r = sqrtl(x); while (r*r > x) r--; while ((r+1)*(r+1) <= x) r++;
   隨機: mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()); shuffle(v.begin(), v.end(), rng);
*/
```

3 資料結構

3.1 BIT (Fenwick Tree)

用法: 單點修改、前綴和。1-based。變形：區間加區間和 (兩棵 BIT)、第 k 小、逆序對。

時間複雜度: $O(\log N)$

```
// 1-based！n = 上界。每筆測資: fill(bit, bit+n+1, 0)
const int MAXN = 200005;
int n;
ll bit[MAXN];
void add(ll *t, int x, ll v) { for (; x <= n; x += x & -x) t[x] += v; }
ll sum(ll *t, int x) { ll r = 0; for (; x > 0; x -= x & -x) r += t[x]; return r; }
// 基本用法: 單點加 add(bit,i,v)；區間和 sum(bit,r) - sum(bit,l-1)

// [變形 1] 區間加 + 單點查: 對差分建 BIT
//   add(bit,l,v); add(bit,r+1,-v);  a[x] = sum(bit,x)
// [變形 2] 區間加 + 區間和: 兩棵 BIT
ll B1[MAXN], B2[MAXN];
void range_add(int l, int r, ll v) {
    add(B1, l, v); add(B1, r + 1, -v);
    add(B2, l, v * (l - 1)); add(B2, r + 1, -v * r);
}
ll pre(int x) { return sum(B1, x) * x - sum(B2, x); }
ll range_sum(int l, int r) { return pre(r) - pre(l - 1); }

// [變形 3] 第 k 小: bit 存「值 x 出現幾次」，回傳最小 x 使 sum(bit,x) >= k
int kth(ll k) {
    int pos = 0;
    for (int pw = 1 << __lg(n); pw; pw >>= 1)
        if (pos + pw <= n && bit[pos + pw] < k) pos += pw, k -= bit[pos];
    return pos + 1; // 回傳 n+1 代表總數 < k
}
// [變形 4] 逆序對: 值先壓縮到 1..n，i=1..n 由左往右:
//   ans += (i-1) - sum(bit, a[i]); add(bit, a[i], 1);   // 前面比 a[i] 大的個數
```

3.2 Segment Tree 線段樹 (lazy)

用法: 區間加值 + 區間和。改成區間 max/min 只要改標 [改] 的 4 行。1-based。

時間複雜度: 建樹 $O(N)$，修改/查詢 $O(\log N)$

```
// 1-based。區間加值 + 區間和 (lazy)。改成 max/min 只要改有 [改] 的行
const int MAXN = 200005;
int n;
ll a[MAXN], t[4 * MAXN], lz[4 * MAXN]; // t: 節點答案, lz: 還沒往下傳的加值

void apply_tag(int id, int l, int r, ll v) {
    t[id] += v * (r - l + 1); // [改] max/min: t[id] += v;
    lz[id] += v;
}
void pull(int id) { t[id] = t[id * 2] + t[id * 2 + 1]; } // [改] max(..)/min(..)
void push(int id, int l, int r) {
    if (lz[id] == 0) return;
    int m = (l + r) / 2;
    apply_tag(id * 2, l, m, lz[id]);
    apply_tag(id * 2 + 1, m + 1, r, lz[id]);
    lz[id] = 0;
}
void build(int id = 1, int l = 1, int r = n) {
    lz[id] = 0;
    if (l == r) { t[id] = a[l]; return; }
    int m = (l + r) / 2;
    build(id * 2, l, m); build(id * 2 + 1, m + 1, r);
    pull(id);
}
void update(int ql, int qr, ll v, int id = 1, int l = 1, int r = n) { // [ql,qr] 加 v
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { apply_tag(id, l, r, v); return; }
    push(id, l, r);
    int m = (l + r) / 2;
    update(ql, qr, v, id * 2, l, m); update(ql, qr, v, id * 2 + 1, m + 1, r);
    pull(id);
}
ll query(int ql, int qr, int id = 1, int l = 1, int r = n) {
    if (qr < l || r < ql) return 0; // [改] 單位元: sum 0, max -INF, min INF
    if (ql <= l && r <= qr) return t[id];
    push(id, l, r);
    int m = (l + r) / 2;
    return query(ql, qr, id * 2, l, m) + query(ql, qr, id * 2 + 1, m + 1, r); // [改]
}
// 用法: 讀 n, a[1..n] → build(); update(l,r,v); query(l,r)

/* [變形]
1. 區間「設值」: 另開 bool has[]、ll st[]
   apply_tag: t[id] = v*(r-l+1) (max: v); st[id] = v; has[id] = 1; lz[id] = 0;
   push: 先下傳 has/st，再下傳 lz。同時有設值 + 加值時，設值會清掉加值
2. 單點設值: update 走到葉子 (l==r) 直接 t[id] = v，免 lazy
3. 需要合併多個資訊 (例: 最大子段和): 開 sum/pre/suf/best 四個陣列,
   pull: sum=L.sum+R.sum, pre=max(L.pre,L.sum+R.pre),
   suf=max(R.suf,R.sum+L.suf),
        best=max({L.best,R.best,L.suf+R.pre})
4. 找第一個 >= x 的位置 (max 樹): 若 t[左] >= x 往左走，否則往右走
5. 值域線段樹: 下標當「值」(先座標壓縮)，t 存個數 → 可求第 k 小 / 區間內個數
*/
```

3.3 Sparse Table 稀疏表

用法: 不修改的區間 max/min/gcd 查詢。0-based。

時間複雜度: 建表 $O(N \log N)$，查詢 $O(1)$

```
// 靜態 (不會修改) 區間 max/min/gcd 查詢。0-based
const int MAXN = 200005, LOG = 18; // 2^LOG > MAXN 即可 (1e6 用 20)
int st[LOG][MAXN];
void build(int n, int *a) {
    for (int i = 0; i < n; i++) st[0][i] = a[i];
    for (int j = 1; (1 << j) <= n; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[j][i] = max(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]); // [改] min/gcd
}
int query(int l, int r) { // [l, r]
    int j = __lg(r - l + 1);
    return max(st[j][l], st[j][r - (1 << j) + 1]); // [改]
}
// 注意: 只能用在「重疊不影響答案」的運算 (max/min/gcd/and/or)，sum 不行
```

3.4 Prefix Sum 前綴和與 Difference 差分

用法: 前綴和: 多次區間查詢；差分: 大量區間加值、最後才查。1D/2D 皆 1-based。

時間複雜度: 建表 $O(N)$，查詢 $O(1)$

```
// 全部 1-based，讓 index 0 當作 0，免判斷邊界
const int MAXN = 1005;
int n, m;
ll a[MAXN], pre[MAXN];                  // 1D
ll A[MAXN][MAXN], P[MAXN][MAXN];        // 2D
ll d[MAXN], D[MAXN][MAXN];              // 差分

void build1() { for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i]; } // [改] XOR: ^
ll query1(int l, int r) { return pre[r] - pre[l - 1]; }
// XOR: pre[r]^pre[l-1]

void build2() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            P[i][j] = P[i - 1][j] + P[i][j - 1] - P[i - 1][j - 1] + A[i][j];
}
ll query2(int x1, int y1, int x2, int y2) { // 左上 (x1,y1) 右下 (x2,y2)
    return P[x2][y2] - P[x1 - 1][y2] - P[x2][y1 - 1] + P[x1 - 1][y1 - 1];
}

// 1D 差分: 大量「區間加」，最後才查
void add1(int l, int r, ll v) { d[l] += v; d[r + 1] -= v; }
void restore1() { for (int i = 1; i <= n; i++) d[i] += d[i - 1]; } // 之後 a[i] += d[i]

// 2D 差分
void add2(int x1, int y1, int x2, int y2, ll v) {
    D[x1][y1] += v; D[x2 + 1][y1] -= v; D[x1][y2 + 1] -= v; D[x2 + 1][y2 + 1] += v;
}
void restore2() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) D[i][j] += D[i - 1][j] + D[i][j - 1] - D[i - 1][j - 1];
}
/* [變形]
- 子陣列和 = k 的個數 (有負數): map<ll,int> cnt; cnt[0]=1; 每步 ans += cnt[pre-k]; cnt[pre]++
- 子陣列和可被 k 整除: 同上，key 改成 ((pre%k)+k)%k
- 最長和為 0 的子陣列: 記錄每個 pre 值「第一次出現」的位置
*/
```

3.5 Monotonic Stack 單調棧

用法: 左/右邊第一個比自己大 (小) 的位置。

時間複雜度: $O(N)$

```cpp
// 堆疊存 index。O(N)。0-based，找不到為 -1
const int MAXN = 200005;
int n, a[MAXN], nxt[MAXN], prv[MAXN];
void next_greater() { // nxt[i] = i 右邊第一個「> a[i]」的位置
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) { // [改] 下一個更小: >
            nxt[st.top()] = i; st.pop();
        }
        st.push(i);
    }
    while (!st.empty()) nxt[st.top()] = -1, st.pop();
}
void prev_greater() { // prv[i] = i 左邊第一個「> a[i]」的位置
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] <= a[i]) st.pop(); // [改] 前一個更小: >=
        prv[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }
}
/* [應用] 直方圖最大矩形: 對每個 i 找左右第一個「更矮」的 L,R，
   面積 = a[i] * (R - L - 1)。「以 a[i] 為最小值的最長區間」都是這招 */
```

3.6 Monotonic Queue 單調隊列

用法: 固定長度視窗的最大/最小值。
時間複雜度: $O(N)$

```cpp
// Sliding Window Maximum: 每個長度 k 的視窗的最大值。O(N)
const int MAXN = 200005;
int n, k, a[MAXN];
vector<int> window_max() {
    deque<int> dq; // 存 index，對應的值由前到後遞減
    vector<int> res;
    for (int i = 0; i < n; i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // 過期
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // [改] 最小值: >=
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
// [應用] DP 優化: dp[i] = max(dp[j]) + c, j in [i-k, i-1] → 對 dp 做 monotonic queue
```

3.7 Disjoint Set 並查集

用法: 合併集合、判斷是否同一組。路徑壓縮 + 小掛大。
時間複雜度: 均攤近似 $O(1)$

```cpp
const int MAXN = 200005;
int f[MAXN], sz[MAXN];
void init(int n) { for (int i = 0; i <= n; i++) f[i] = i, sz[i] = 1; }
int find(int x) { return f[x] == x ? x : f[x] = find(f[x]); } // 路徑壓縮
bool unite(int a, int b) { // 回傳是否真的合併 (Kruskal 判環用)
    a = find(a), b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b); // 小掛大
    f[b] = a; sz[a] += sz[b];
    return true;
}
/* [變形]
1. 集合大小: sz[find(x)]；集合個數: 一開始 n，每次 unite 成功就 -1
2. 帶權 DSU (x 到根的距離/差值 w[x])，find 時累加:
   int find(int x){ if(f[x]==x) return x; int r=find(f[x]); w[x]+=w[f[x]];
   return f[x]=r; }
   (此時 find 不能先改 f[x]：要先存 root 再加 w，如上)
3. 二分圖/敵人關係: 開 2n 個點，x 與 y 敵對 → unite(x, y+n), unite(x+n, y)
4. 「下一個可用位置」(跳過已刪除): 刪 i 時 f[i] = i+1，find(i) 即右邊第一個沒刪的
5. 離線刪邊 → 反過來變成加邊
*/
```

3.8 座標壓縮、pb_ds 平衡樹

用法: 值很大但個數少 → 壓到 $1 \sim k$。ordered_set 可查第 k 小與名次。
時間複雜度: $O(N \log N)$

```cpp
// 座標壓縮: 值很大 (1e9) 但個數少 → 映射到 1..k，之後可開陣列/BIT
vector<ll> vals;
void compress(vector<ll> &a) {
    vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (ll &x : a) x = lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1;
} // 還原: vals[id - 1]

// pb_ds 平衡樹 (可查名次的 set)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<ll, null_type, less<ll>, rb_tree_tag,
tree_order_statistics_node_update> ordered_set;
// ordered_set os; os.insert(x); os.erase(x);
// *os.find_by_order(k): 第 k 小 (0-based)；os.order_of_key(x): 有幾個 < x
// 要允許重複值: 改存 pair<ll,int>{值, 獨一無二的編號}
```

4 數學

4.1 快速冪

用法: 計算 $a^b \bmod m$。
時間複雜度: $O(\log b)$

```cpp
// (a^b) % mod，mod 常為 1e9+7 或 998244353
ll fast_pow(ll a, ll b, ll mod) {
    ll res = 1; a %= mod;
    if (a < 0) a += mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;   // mod > 3e9 時改 (__int128)res * a % mod
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
// 費馬小定理降冪: p 質數、gcd(a,p)=1 時 a^b = a^(b mod (p-1)) (mod p)
// 歐拉降冪 (一般 m): b >= phi(m) 時 a^b = a^(b mod phi(m) + phi(m)) (mod m)
```

4.2 模逆元 (費馬小定理)

用法: p 為質數且 $p \nmid a$：$a^{-1} \equiv a^{p-2} \pmod p$，所以 $x/a \equiv x \cdot a^{p-2}$。另附 $O(N)$ 建逆元表。
時間複雜度: $O(\log p)$

```cpp
// 需要 fast_pow。p 必須是質數且 a % p != 0；不是質數請用 exGCD 版
ll inv(ll a, ll p) { return fast_pow(a, p - 2, p); }
ll mod_div(ll a, ll b, ll p) { return (a % p + p) % p * inv(b, p) % p; } // (a/b) % p

// O(N) 建 1~n 的逆元表 (p 質數, n < p)
const int MAXN = 1000005;
ll invt[MAXN];
void build_inv(int n, ll p) {
    invt[1] = 1;
    for (int i = 2; i <= n; i++) invt[i] = (p - p / i) * invt[p % i] % p;
}
// 減法取模記得: ((a - b) % p + p) % p
```

4.3 擴展歐幾里得

用法: 解 $ax + by = \gcd(a, b)$；模數不是質數時的模逆元。
時間複雜度: $O(\log \min(a, b))$

```cpp
// 求 ax + by = gcd(a,b) 的一組解，回傳 gcd
ll extgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll g = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
/* 解 ax + by = c: g = gcd(a,b)，c % g != 0 → 無解
   否則 x0 = x*(c/g), y0 = y*(c/g)；通解 x = x0 + k*(b/g), y = y0 - k*(a/g)
   最小非負 x: t = b/g; x = (x0 % t + t) % t  (注意 x*(c/g) 可能溢位 → __int128) */

// 模逆元 (m 不必是質數)，不存在回傳 -1
ll inverse(ll a, ll m) {
    ll x, y;
    if (extgcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}
```

4.4 組合數 C(n,k)

用法: $n \le 2000$ 用帕斯卡 (模數可非質數)；$n \le 10^6$ 用階乘 + 逆元 (模數須為質數)；n 到 10^{18}、質數模數小用 Lucas。
時間複雜度: $O(N^2)$ / $O(N)$ 建表、$O(1)$ 查詢

```cpp
const ll MOD = 1e9 + 7; // [改] 依題目
// 一、帕斯卡三角形 (n <= 5000)：只有加法，MOD 不是質數也能用
const int MAXC = 2005;
int C[MAXC][MAXC];
void build_pascal() {
    for (int i = 0; i < MAXC; i++) {
        C[i][0] = 1;
        for (int j = 1; j <= i; j++) C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
    }
}
// 二、階乘 + 逆元 (n <= 1e6)，MOD 必須是質數。需要 fast_pow
const int MAXN = 1000005;
ll fact[MAXN], ifact[MAXN];
void build_fact() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
    ifact[MAXN - 1] = fast_pow(fact[MAXN - 1], MOD - 2, MOD);
    for (int i = MAXN - 1; i > 0; i--) ifact[i - 1] = ifact[i] * i % MOD;
}
ll nCr(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * ifact[k] % MOD * ifact[n - k] % MOD;
}
// 三、Lucas: n,k 很大 (1e18) 但質數 p 小 (<= 1e6)。要先把 MOD 設成 p 再 build_fact
ll lucas(ll n, ll k, ll p) {
    if (k == 0) return 1;
    return nCr(n % p, k % p) * lucas(n / p, k / p, p) % p;
}
// 四、n 小 k 小但不取模: C(n,k) 用 res = res * (n-i) / (i+1) 逐步算 (先乘後除會整除)
```

4.5 質數、質因數分解、線性篩

用法: 單一數字 $O(\sqrt{N})$；大量查詢用線性篩 (順便得到最小質因數 spf)。
時間複雜度: $O(\sqrt{N})$ / 篩 $O(N)$

```
// 一、單一數字判斷 / 分解，O(sqrt N)，N <= 1e12 可用 (更大用 Miller-Rabin)
bool is_prime(ll n) {
    if (n < 2) return false;
    for (ll i = 2; i * i <= n; i++) if (n % i == 0) return false;
    return true;
}
vector<pair<ll, int>> factorize(ll n) { // {質因數, 次方}
    vector<pair<ll, int>> res;
    for (ll i = 2; i * i <= n; i++) {
        if (n % i) continue;
        int c = 0;
        while (n % i == 0) n /= i, c++;
        res.push_back({i, c});
    }
    if (n > 1) res.push_back({n, 1});
    return res;
}
// 二、Euler 線性篩 O(N): 質數表 + 最小質因數 spf，N <= 1e7
const int MAXN = 10000005;
int spf[MAXN];
vector<int> primes; // 1e4: 1229, 1e5: 9592, 1e6: 78498, 1e7: 664579
void sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) spf[i] = i, primes.push_back(i);
        for (int p : primes) {
            if (p > spf[i] || (ll)i * p > n) break;
            spf[i * p] = p;
        }
    }
}
// 有 spf 後，分解 x 只要 O(log x): while(x > 1){ int p = spf[x]; x /= p; ... }
// 因數個數 = Π(e_i + 1)；因數和 = Π(p^(e+1)-1)/(p-1)
// 區間篩 [L,R] (R 到 1e12, R-L 到 1e6): 用 sqrt(R) 內的質數去劃掉 [L,R] 內的倍數

/* 哥德巴赫: N 最少拆成幾個質數的和
   N 是質數 → 1；N 偶數 (N>2) → 2；N 奇數且 N-2 是質數 → 2；其他 → 3 */
```

範圍	擁有最多因數的數	因數個數
10^9	735134400	1344
2^{31}	2095133040	1600
10^{18}	897612484786617600	103680

4.6 尤拉函數 φ

用法: $1 \sim N$ 中與 N 互質的個數；另附 $1 \sim N$ 全部建表。
時間複雜度: $O(\sqrt{N})$ / 建表 $O(N \log\log N)$

```
// phi(n) = 1~n 中與 n 互質的個數，O(sqrt N)
ll phi(ll n) {
    ll res = n;
    for (ll i = 2; i * i <= n; i++) {
        if (n % i) continue;
        while (n % i == 0) n /= i;
        res -= res / i; // res *= (1 - 1/i)
    }
    if (n > 1) res -= res / n;
    return res;
}
// 1~n 全部的 phi，O(N log log N)
const int MAXN = 1000005;
int ph[MAXN];
void phi_table(int n) {
    for (int i = 0; i <= n; i++) ph[i] = i;
    for (int i = 2; i <= n; i++)
        if (ph[i] == i) // i 是質數
            for (int j = i; j <= n; j += i) ph[j] -= ph[j] / i;
}
// 性質: Σ_{d|n} phi(d) = n；1~n 中與 n 互質的數總和 = n*phi(n)/2 (n>1)
// gcd(i,n)=d 的 i 個數 = phi(n/d) → Σ gcd(i,n) = Σ_{d|n} d*phi(n/d)
```

4.7 Legendre's Formula 勒讓德公式

用法: $n!$ 中質數 p 的次數 $v_p(n!) = \sum_{k\geq 1}\lfloor n/p^k \rfloor$；$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!)$。
時間複雜度: $O(\log_p n)$

```
// n! 中質數 p 的次數 = Σ floor(n / p^k)，O(log_p n)
ll legendre(ll n, ll p) {
    ll ans = 0;
    while (n) n /= p, ans += n;
    return ans;
}
// C(n,k) 中 p 的次數
ll c_vp(ll n, ll k, ll p) { return legendre(n, p) - legendre(k, p) - legendre(n - k, p); }
/* 用途: n! 尾端 0 的個數 = legendre(n, 5)
   m 是否整除 n!: 把 m 質因數分解，每個 p^e 檢查 legendre(n,p) >= e
   n! 在 b 進位的尾 0: b = Π p^e → min( legendre(n,p) / e ) */
```

4.8 數論分塊

用法: 快速計算 $\sum_{i=1}^{n}\lfloor n/i \rfloor$ ($\lfloor n/i \rfloor$ 只有 $O(\sqrt{n})$ 種值)。
時間複雜度: $O(\sqrt{N})$

```
// Σ_{i=1..n} floor(n/i)，O(sqrt N)。同一個區塊 [l, r] 的 n/i 都相同
ll math_block(ll n) {
    ll ans = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);          // 與 l 商相同的最右邊界
        ans += (r - l + 1) * (n / l); // [改] Σ f(i)*floor(n/i): 改成 (F(r)-F(l-1))*(n/l)，F 為 f 的前綴和
    }
    return ans;
}
// 兩個: Σ floor(n/i)*floor(m/i) → r = min(n/(n/l), m/(m/l))，l 跑到 min(n,m)
// 只要 i 在 1..k (k<n): r = min(r, k)
```

4.9 排容原理

用法: 「至少滿足一個條件」的計數，奇加偶減。
時間複雜度: $O(2^M M)$

```
// 排容: 1~n 中能被 ps 中「至少一個」整除的個數。O(2^m * m)
ll inclusion_exclusion(ll n, const vector<ll> &ps) {
    int m = ps.size();
    ll ans = 0;
    for (int mask = 1; mask < (1 << m); mask++) {
        ll l = 1; int bits = 0;
        for (int i = 0; i < m && l <= n; i++)
            if (mask >> i & 1) {
                bits++;
                l = (l > n / ps[i]) ? n + 1 : l * ps[i]; // [改] 不是質數要用 lcm；防溢位
            }
        if (l > n) continue;
        ans += (bits & 1) ? n / l : -(n / l); // 奇加偶減
    }
    return ans;
}
/* 1~n 與 K 互質的個數 = n - inclusion_exclusion(n, K 的質因數)
   錯排 D(n) = (n-1)(D(n-1)+D(n-2))，D(1)=0, D(2)=1
   「每個條件都不滿足」= 總數 - 至少滿足一個；「恰好 k 個」可用二項式反演 */
```

4.10 矩陣快速冪

用法: 線性遞迴 $f(n) = \sum c_i f(n-i)$ 在 n 很大時。含取模。
時間複雜度: $O(k^3 \log n)$

```
// 線性遞迴 f(n) = c1 f(n-1) + ... + ck f(n-k)，n 到 1e18 → 矩陣快速冪 O(k^3 log n)
const ll MOD = 1e9 + 7;
typedef vector<vector<ll>> Mat;
Mat mul(const Mat &A, const Mat &B) { // (n x k) * (k x m)
    int n = A.size(), k = B.size(), m = B[0].size();
    Mat C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) if (A[i][t])
            for (int j = 0; j < m; j++) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
    return C;
}
Mat mpow(Mat A, ll p) { // A 必須是方陣
    int n = A.size();
    Mat R(n, vector<ll>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1; // 單位矩陣
    for (; p; p >>= 1, A = mul(A, A)) if (p & 1) R = mul(R, A);
    return R;
}
/* 例: Fibonacci  [F(n+1)]   [1 1]^n [F(1)]
                  [F(n)  ] = [1 0]   [F(0)]  → F(n) = mpow({{1,1},{1,0}}, n)[1][0]
   一般 k 階: 第一列放係數 c1..ck，下面是單位矩陣往下平移一格
   有常數項 +d: 狀態向量多放一個 1，矩陣多一行一列
   圖上「恰好走 k 步」的路徑數 = 鄰接矩陣 ^k；(min,+) 版本可求恰好 k 步最短路 */
```

4.11 高斯消去、XOR 線性基

用法: 解聯立方程組 (判斷無解/唯一/無限多解)；一堆數選子集的最大 XOR。
時間複雜度: $O(N^3)$ / $O(\log V)$ 每次插入

```
// 高斯消去解 n 元一次方程組 A x = b (浮點數)。a[i][n] 放 b[i]
// 回傳: 0 無解, 1 唯一解 (存於 x), 2 無限多解
const double EPS = 1e-9;
int gauss(vector<vector<double>> a, vector<double> &x) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; col++) {
        int sel = row;
        for (int i = row; i < n; i++) if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        if (fabs(a[sel][col]) < EPS) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        for (int i = 0; i < n; i++) if (i != row) {
            double c = a[i][col] / a[row][col];
            for (int j = col; j <= m; j++) a[i][j] -= a[row][j] * c;
        }
        row++;
    }
    x.assign(m, 0);
    for (int i = 0; i < m; i++) if (where[i] != -1) x[i] = a[where[i]][m] / a[where[i]][i];
    for (int i = 0; i < n; i++) {
        double s = 0;
        for (int j = 0; j < m; j++) s += x[j] * a[i][j];
        if (fabs(s - a[i][m]) > EPS) return 0;
    }
    for (int i = 0; i < m; i++) if (where[i] == -1) return 2;
    return 1;
}
// 模質數版: 除法改成乘逆元、fabs>EPS 改成 != 0；XOR 版 (開關燈問題): 減法改 ^，用 bitset

// XOR 線性基: 從一堆數選子集，XOR 最大值 / 能否湊出 x
ll basis[64];
```

```cpp
bool insert_xor(ll x) {
    for (int i = 63; i >= 0; i--) if (x >> i & 1) {
        if (!basis[i]) { basis[i] = x; return true; }
        x ^= basis[i];
    }
    return false; // x 已能被湊出
}
ll max_xor() { ll r = 0; for (int i = 63; i >= 0; i--) r = max(r, r ^ basis[i]); return r; }
```

4.12 大數運算

用法: 加、減、比較、乘小數、乘大數、除以小數。先確認 ___int128 夠不夠。

時間複雜度: 乘法 $O(NM)$，其餘 $O(N)$

```cpp
// 大數: vector<int> 反向存 (個位在 [0])。先想想 __int128 (到 1.7e38) 夠不夠
typedef vector<int> Big;
Big to_big(const string &s) { Big a; for (int i = s.size() - 1; i >= 0; i--) a.push_back(s[i] - '0'); return a; }
void trim(Big &a) { while (a.size() > 1 && a.back() == 0) a.pop_back(); }
void print(const Big &a) { for (int i = a.size() - 1; i >= 0; i--) cout << a[i]; cout << '\n'; }
int cmp(const Big &a, const Big &b) { // -1: a<b, 0: a==b, 1: a>b
    if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
    for (int i = a.size() - 1; i >= 0; i--) if (a[i] != b[i]) return a[i] < b[i] ? -1 : 1;
    return 0;
}
Big add(const Big &a, const Big &b) {
    Big c; int carry = 0;
    for (size_t i = 0; i < max(a.size(), b.size()) || carry; i++) {
        if (i < a.size()) carry += a[i];
        if (i < b.size()) carry += b[i];
        c.push_back(carry % 10); carry /= 10;
    }
    return c;
}
Big sub(const Big &a, const Big &b) { // 需 a >= b
    Big c = a; int borrow = 0;
    for (size_t i = 0; i < c.size(); i++) {
        c[i] -= borrow + (i < b.size() ? b[i] : 0);
        borrow = c[i] < 0;
        if (borrow) c[i] += 10;
    }
    trim(c); return c;
}
Big mul_small(const Big &a, ll b) { // b 到 1e9 都可
    Big c; ll carry = 0;
    for (size_t i = 0; i < a.size() || carry; i++) {
        if (i < a.size()) carry += a[i] * b;
        c.push_back(carry % 10); carry /= 10;
    }
    trim(c); return c;
}
Big mul(const Big &a, const Big &b) { // O(N*M)
    vector<ll> c(a.size() + b.size(), 0);
    for (size_t i = 0; i < a.size(); i++)
        for (size_t j = 0; j < b.size(); j++) c[i + j] += a[i] * b[j];
    Big r; ll carry = 0;
    for (size_t i = 0; i < c.size(); i++) { carry += c[i]; r.push_back(carry % 10); carry /= 10; }
    trim(r); return r;
}
Big div_small(const Big &a, ll b, ll &rem) { // 商與餘數
    Big c(a.size()); rem = 0;
    for (int i = a.size() - 1; i >= 0; i--) { rem = rem * 10 + a[i]; c[i] = rem / b; rem %= b; }
    trim(c); return c;
}
```

4.13 博弈論 (Nim、SG 函數)

時間複雜度: $O(N \cdot |\text{moves}|)$

```cpp
/* 博弈 (兩人輪流、無法行動者輸)
- Nim: 多堆石頭任取 → a1 ^ a2 ^ ... ^ an != 0 先手勝
- Bash: 一堆 n 個，每次取 1~k → n % (k+1) != 0 先手勝
- 階梯 Nim: 只看奇數階的 XOR
- 不確定時: 小範圍暴力算 win[] 找規律 (打表!)
- 多個獨立遊戲的組合 → 各自算 SG 值再 XOR，!= 0 先手勝 */
const int MAXN = 100005;
int sg[MAXN];
vector<int> moves = {1, 3, 4}; // [改] 每次可拿的數量
void build_sg(int n) {
    for (int i = 0; i <= n; i++) {
        set<int> s; // 所有後繼狀態的 SG 值
        for (int m : moves) if (i >= m) s.insert(sg[i - m]);
        int g = 0;
        while (s.count(g)) g++; // mex
        sg[i] = g;
    }
}
// sg[i] == 0 → 必敗態 (P-position)；!= 0 → 必勝態
```

4.14 常用公式

- Catalan: $C_n = \frac{1}{n+1}\binom{2n}{n}$，1, 1, 2, 5, 14, 42, 132。合法括號、二元樹個數、不越過對角線的路徑。
- 第二類 Stirling (n 個相異球放進 k 個相同非空盒): $S(n,k) = kS(n-1,k) + S(n-1,k-1)$。
- 錯排: $D_n = (n-1)(D_{n-1} + D_{n-2})$，$D_1 = 0, D_2 = 1$。
- 插板法: $x_1 + \cdots + x_k = n$ 的非負整數解 $\binom{n+k-1}{k-1}$；正整數解 $\binom{n-1}{k-1}$。
- 格子路徑 $(0,0) \to (a,b)$ 只往右/上: $\binom{a+b}{a}$。
- $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$，$\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$，等比 $\sum_{i=0}^{n-1} r^i = \frac{r^n-1}{r-1}$。
- 費馬: $a^{p-1} \equiv 1$；尤拉: $a^{\varphi(m)} \equiv 1 \ (\gcd(a,m) = 1)$；Wilson: $(p-1)! \equiv -1 \pmod p$。
- $\gcd(a,b) = \gcd(b, a \bmod b)$，$\text{lcm}(a,b) = a/\gcd(a,b) \cdot b$ (先除再乘防溢位)。
- 平面圖 Euler: $V - E + F = 2$ (連通，F 含外部面)。Pick: $A = I + B/2 - 1$。

5 字串

5.1 KMP 字串匹配

用法: `pi[i]` $= s[0..i]$ 最長「相等真前後綴」的**長度**。回傳所有 0-based 起點；也可求最小循環節。

時間複雜度: $O(N+M)$

```cpp
// pi[i] = s[0..i] 最長「相等真前後綴」的長度
vector<int> prefix_function(const string &s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1]; // 失配就往回跳
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
// p 在 t 中所有出現的起點 (0-based)，O(|t| + |p|)
vector<int> kmp(const string &t, const string &p) {
    vector<int> pi = prefix_function(p), res;
    for (int i = 0, j = 0; i < (int)t.size(); i++) {
        while (j > 0 && t[i] != p[j]) j = pi[j - 1];
        if (t[i] == p[j]) j++;
        if (j == (int)p.size()) { res.push_back(i - j + 1); j = pi[j - 1]; }
    }
    return res;
}
// 最小循環節: k = n - pi[n-1]。n % k == 0 → s 由 s.substr(0,k) 重複 n/k 次；否則答案是 n
int min_period(const string &s) {
    int n = s.size(), k = n - prefix_function(s).back();
    return n % k == 0 ? k : n;
}
/* [變形]
- 不重疊計數: 匹配成功後 j = 0 (而不是 pi[j-1])
- s 的所有「既是前綴也是後綴」的長度: j = pi[n-1], 一直 j = pi[j-1] 直到 0
- 最短補字元變迴文 (補在後面): t = rev(s) + '#' + s，答案 n - pi.back()
*/
```

5.2 Z-Algorithm

用法: `z[i]` $= s$ 與 $s[i..]$ 的最長共同前綴。0-based。

時間複雜度: $O(N)$

```cpp
// z[i] = s 與 s[i..] 的最長共同前綴長度 (z[0] 定義為 0)，O(N)
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) { // [l, r] 為目前最右的匹配區間
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}
// 字串匹配: s = p + '$' + t ('$' 不能出現在 p,t 中)
vector<int> z_match(const string &t, const string &p) {
    vector<int> z = z_function(p + '$' + t), res;
    for (int i = p.size() + 1; i < (int)z.size(); i++)
        if (z[i] == (int)p.size()) res.push_back(i - p.size() - 1);
    return res;
}
// 最小週期: 最小的 i 使 i + z[i] == n (且 n % i == 0 才是整除循環)
```

5.3 Manacher 迴文

用法: 最長迴文子字串、迴文子字串總數、$O(1)$ 判斷 $s[l..r]$ 是否迴文。

時間複雜度: $O(N)$

```cpp
// 把 "abc" 變成 "$#a#b#c#^"，奇偶長度統一處理。p[i] = 以 t[i] 為中心的半徑
// p[i] 剛好等於「原字串中對應迴文的長度」，起點 = (i - p[i]) / 2
string t;
vector<int> p;
void manacher(const string &s) {
    t = "$#";
    for (char c : s) t += c, t += '#';
    t += '^';
    int n = t.size(), c = 0, r = 0;
    p.assign(n, 0);
    for (int i = 1; i < n - 1; i++) {
        if (i < r) p[i] = min(r - i, p[2 * c - i]);              // 抄鏡像
        while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) p[i]++;       // 暴力擴展
        if (i + p[i] > r) c = i, r = i + p[i];
    }
}
string longest_palindrome(const string &s) {
    manacher(s);
    int best = 1;
    for (int i = 1; i + 1 < (int)t.size(); i++) if (p[i] > p[best]) best = i;
```

```
    return s.substr((best - p[best]) / 2, p[best]);
}
ll count_palindromes(const string &s) { // 迴文子字串總數（位置不同算不同）
    manacher(s);
    ll total = 0;
    for (int i = 1; i + 1 < (int)t.size(); i++) total += (p[i] + 1) / 2;
    return total;
}
// s[l..r] 是否迴文：中心 i = l + r + 2，檢查 p[i] >= r - l + 1
```

5.4 Trie 字典樹

用法： 字串出現次數、前綴個數、刪除；SIGMA=2 時可做最大 XOR。root = 0。

時間複雜度： $O(|S|)$

```
// root = 0，節點從 1 開始編號；tr[u][c] == 0 代表沒有這條路
const int MAXN = 1000005, SIGMA = 26; // MAXN = 所有字串總長 + 1；[改] 二進位 XOR: SIGMA = 2
int tr[MAXN][SIGMA], pass_cnt[MAXN], end_cnt[MAXN], cnt = 0;
void trie_init() { // 多筆測資：只清用過的節點
    for (int i = 0; i <= cnt; i++) memset(tr[i], 0, sizeof tr[i]), pass_cnt[i] = end_cnt[i] = 0;
    cnt = 0;
}
void insert(const string &s) {
    int u = 0; pass_cnt[u]++;
    for (char ch : s) {
        int c = ch - 'a';          // [改] 字元集
        if (!tr[u][c]) tr[u][c] = ++cnt;
        u = tr[u][c]; pass_cnt[u]++;
    }
    end_cnt[u]++;
}
int walk(const string &s) { // 走到 s 的節點，不存在回傳 -1
    int u = 0;
    for (char ch : s) { u = tr[u][ch - 'a']; if (!u) return -1; }
    return u;
}
int count_word(const string &s) { int u = walk(s); return u == -1 ? 0 : end_cnt[u]; }   // s 出現幾次
int count_prefix(const string &s) { int u = walk(s); return u == -1 ? 0 : pass_cnt[u]; } // 以 s 為前綴的個數
void erase(const string &s) { // 需確定 s 存在
    int u = 0; pass_cnt[u]--;
    for (char ch : s) { u = tr[u][ch - 'a']; pass_cnt[u]--; }
    end_cnt[u]--;
}
// [變形] 最大 XOR 配對：SIGMA=2，把數字從高位 (bit 30 → 0) 插入；
// 查 x 時每一位盡量走「相反」的 bit: int b = x>>i&1; if(tr[u][!b] && pass_cnt[tr[u][!b]]) res|=1<<i, u=tr[u][!b]; else u=tr[u][b];
```

5.5 AC 自動機

用法： 多個 pattern 同時匹配，求每個 pattern 在文章中出現幾次。fail[v] 指向 v 的最長且存在於 Trie 中的真後綴。

時間複雜度： $O(\text{總長} \cdot \Sigma + |T|)$

```
// 多個 pattern 同時在文章中匹配。root = 0
const int MAXN = 1000005, SIGMA = 26; // MAXN = pattern 總長 + 1
int tr[MAXN][SIGMA], fail_[MAXN], occ[MAXN], cnt = 0;
vector<int> order_; // BFS 順序
int insert(const string &s) { // 回傳 s 的結尾節點（記下來查答案用）
    int u = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (!tr[u][c]) tr[u][c] = ++cnt;
        u = tr[u][c];
    }
    return u;
}
void build() { // 全部 insert 完再呼叫
    queue<int> q;
    for (int c = 0; c < SIGMA; c++) if (tr[0][c]) fail_[tr[0][c]] = 0, q.push(tr[0][c]);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order_.push_back(u);
        for (int c = 0; c < SIGMA; c++) {
            int v = tr[u][c];
            if (v) fail_[v] = tr[fail_[u]][c], q.push(v);
            else tr[u][c] = tr[fail_[u]][c]; // 不存在的路直接連到 fail 的對應點
        }
    }
}
// 文章 text 跑一次。之後 pattern i 出現次數 = occ[end_node[i]]，O(|text| + 總長)
void query(const string &text) {
    int u = 0;
    for (char ch : text) u = tr[u][ch - 'a'], occ[u]++;
    for (int i = (int)order_.size() - 1; i >= 0; i--) { // 由深到淺，把次數加到 fail
        int v = order_[i];
        occ[fail_[v]] += occ[v];
    }
}
/* [變形]
- 只問「有幾個 pattern 出現過」：看 occ[end_node[i]] > 0 的個數
- 文章不能包含任何 pattern（禁止字串 DP）：先把 bad[v] |= bad[fail[v]]（按 BFS 順序），
  dp[長度][AC 節點]，轉移 v = tr[u][c]，跳過 bad[v] 的狀態
*/
```

6 圖論

表示圖：鄰接串列 `vector<int> g[MAXN]`（帶權用 `pair<int,ll>`）、鄰接矩陣（Floyd、$V \le 500$）、邊列表（Kruskal）。注意：有向/無向、有沒有權重、是否連通、自環與重邊。連通平面圖的區域數可用 $V - E + F = 2$。

6.1 BFS、拓樸排序、網格 BFS、0-1 BFS

用法： 無權最短路；拓樸排序（可判環、DAG DP）；網格四方向；邊權只有 0/1 時用 deque。

時間複雜度： $O(V + E)$

```
const int MAXN = 200005;
int n, m, indeg[MAXN], dist_[MAXN];
vector<int> g[MAXN]; // 多筆測資：for(i=0..n) g[i].clear(), indeg[i]=0;

// 1. 拓樸排序（Kahn）。回傳順序，size < n 代表有環
vector<int> topo_sort() {
    queue<int> q; vector<int> res;          // [改] 要字典序最小：priority_queue<int,vector<int>,greater<int>>
    for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); res.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return res;
}
// DAG 上 DP：依拓樸順序轉移，例如最長路 dp[v] = max(dp[v], dp[u] + w)、路徑數 dp[v] += dp[u]

// 2. BFS：無權圖最短路（邊權全為 1）
void bfs(int s) {                           // [改] 多源 BFS：把所有起點一開始都 push、dist = 0
    fill(dist_, dist_ + n + 1, -1);
    queue<int> q; q.push(s); dist_[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (dist_[v] == -1) dist_[v] = dist_[u] + 1, q.push(v);
    }
}
// 3. 網格 BFS
const int MAXR = 1005;
int R, C, gd[MAXR][MAXR];
char grid[MAXR][MAXR];
int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1}; // [改] 8 方向 / 騎士走法
void grid_bfs(int sx, int sy) {
    for (int i = 0; i < R; i++) fill(gd[i], gd[i] + C, -1);
    queue<pair<int, int>> q; q.push({sx, sy}); gd[sx][sy] = 0;
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx < 0 || nx >= R || ny < 0 || ny >= C) continue;
            if (grid[nx][ny] == '#' || gd[nx][ny] != -1) continue; // [改] 障礙物條件
            gd[nx][ny] = gd[x][y] + 1; q.push({nx, ny});
        }
    }
}
// 4. 0-1 BFS：邊權只有 0 或 1，用 deque，權 0 推前面、權 1 推後面，O(V+E)
vector<pair<int, int>> wg[MAXN];
int d01[MAXN];
void bfs01(int s) {
    fill(d01, d01 + n + 1, INT_MAX);
    deque<int> dq; dq.push_back(s); d01[s] = 0;
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : wg[u]) if (d01[u] + w < d01[v]) {
            d01[v] = d01[u] + w;
            w ? dq.push_back(v) : dq.push_front(v);
        }
    }
}
// 例：網格「最少拆幾道牆」/「最少轉向次數」→ 走空地權 0、拆牆/轉向權 1
```

6.2 DFS

用法： 如果解點數 $N \le 25$，可以直接 dfs 找答案。連通塊、有向圖判環（三色）、小 N 枚舉所有路徑（回溯）。

時間複雜度： $O(V + E)$

```
const int MAXN = 200005;
int n, comp[MAXN], color_[MAXN];
vector<int> g[MAXN];
// 1. 連通塊：comp[v] = 所屬編號（0 = 未拜訪）
void dfs(int u, int id) {
    comp[u] = id;
    for (int v : g[u]) if (!comp[v]) dfs(v, id);
}
int count_components() {
    fill(comp, comp + n + 1, 0);
    int id = 0;
    for (int i = 1; i <= n; i++) if (!comp[i]) dfs(i, ++id);
    return id;
}
// 2. 有向圖找環：0 未拜訪，1 在遞迴堆疊中，2 已完成。走到狀態 1 的點 = 有環
bool has_cycle(int u) {
    color_[u] = 1;
    for (int v : g[u]) {
        if (color_[v] == 1) return true;
        if (color_[v] == 0 && has_cycle(v)) return true;
    }
    color_[u] = 2;
    return false;
}
// 無向圖找環：走到「已拜訪且不是父節點」的點即有環；或 DSU unite 失敗
```

```
// 3. 小 N (<= 10) 枚舉所有路徑: 回溯
bool vis[MAXN];
ll best;
void all_paths(int u, ll len) {
    best = max(best, len);                          // [改] 依題意更新答案 (最長/最短/計數)
    for (int v : g[u]) if (!vis[v]) {
        vis[v] = true; all_paths(v, len + 1);
        vis[v] = false;                             // 回溯: 一定要還原
    }
}
// 遞迴太深 (鏈狀 1e5 以上) 可能 stack overflow → 改用 stack 的迭代版或 BFS
```

6.3 Dijkstra 單源最短路

用法: 邊權 ≥ 0。附路徑還原、最短路條數、瓶頸路寫法與常見變形。
時間複雜度: $O((V+E)\log V)$

```
// 單源最短路，邊權 >= 0。O((V+E) log V)
const int MAXN = 200005;
const ll INF = 1e18;
int n, par[MAXN];
ll dist[MAXN], ways[MAXN];
vector<pair<int, ll>> g[MAXN]; // g[u] = {v, w}；無向圖兩邊都加
void dijkstra(int s) {          // [改] 多源: 所有起點 dist = 0 並 push
    fill(dist, dist + n + 1, INF);
    fill(ways, ways + n + 1, 0);
    fill(par, par + n + 1, -1);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq; // {距離, 點}
    dist[s] = 0; ways[s] = 1; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // 過期的資料
        for (auto [v, w] : g[u]) {
            ll nd = d + w;          // [改] 瓶頸路 (路徑上最大邊最小): nd = max(d, w)
            if (nd < dist[v]) {
                dist[v] = nd; par[v] = u; ways[v] = ways[u];
                pq.push({nd, v});
            } else if (nd == dist[v]) {
                ways[v] = (ways[v] + ways[u]) % 1000000007; // 最短路條數 (邊權需 > 0)
            }
        }
    }
}
vector<int> get_path(int t) { // 先確認 dist[t] != INF
    vector<int> path;
    for (int v = t; v != -1; v = par[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    return path;
}
/* [常見變形]
- 所有點到「同一個終點」: 建反向圖，從終點跑一次
- 必經某點 k: dist(s,k) + dist(k,t)，從 s 和 k 各跑一次
- 某條邊 (u,v,w) 是否在某條最短路上: ds[u] + w + dt[v] == ds[t] (ds 從 s, dt 從 t 在反向圖)
- 點也有權重: 把點權加到「進入該點的邊」上，或拆點 (v_in → v_out 權 = 點權)
- 多一個條件 (次數/時間/燃料/鑰匙...) → 看「分層圖 / 狀態圖」那一節
- 第 K 短路 (可重複點): 每個點最多被 pop K 次，第 K 次 pop 到終點即答案
*/
```

6.4 分層圖 / 狀態圖 (最短路 + 額外條件)

用法: 題目多一個條件時，把「(點, 條件)」當成新的點。方法一直接建大圖再呼叫原本的 Dijkstra；方法二把條件放進 dist 的第二維。
時間複雜度: $O(K(V+E)\log(KV))$

```
/* 分層圖 / 狀態圖：題目在最短路上「多一個條件」時用
   核心: 狀態 = (點, 額外資訊)。把每個狀態當成一個新的點，演算法本身不用改！
   例 (NCPC 型): 最多 K 次可以讓一條邊免費 → 狀態 (v, 已用次數 k)
     (u,k) → (v,k)   權 w   : 正常走
     (u,k) → (v,k+1) 權 0   : 用一次特權 (k < K)
   答案 = min over k of dist(t, k)
*/
// 方法一 (推薦): 直接建出大圖，然後呼叫原本的 Dijkstra (Dijkstra 的 MAXN 要開 >= N*(K+1)+1)
int N, K; // 原圖點數 (1-based)、特權次數
int id(int v, int k) { return k * N + v; }
void build_layered(const vector<array<ll, 3>> &edges) { // 原圖的邊 {u, v, w}
    n = N * (K + 1); // Dijkstra 用的 n 變成「狀態總數」
    for (int i = 0; i <= n; i++) g[i].clear();
    for (auto [u, v, w] : edges)
        for (int k = 0; k <= K; k++) {
            g[id(u, k)].push_back({id(v, k), w});                // [改] 同層轉移
            if (k < K) g[id(u, k)].push_back({id(v, k + 1), 0}); // [改] 跨層轉移 (折半: w/2)
            // 無向圖: 反方向 (v→u) 也要加同樣兩條
        }
}
// 用法: build_layered(edges); dijkstra(id(s, 0));
//       ll ans = INF; for (k = 0..K) ans = min(ans, dist[id(t, k)]);  [改] 必須恰好用 K 次 → 只看 k = K

// 方法二: 狀態放在 dist 的第二維 (狀態太多、或轉移是「當場算出來」時，例如網格 + 鑰匙)
const int MAXV = 100005, MAXK = 11;
ll ds[MAXV][MAXK];
vector<pair<int, ll>> adj[MAXV];
void state_dijkstra(int s) {
    for (int i = 0; i <= N; i++) fill(ds[i], ds[i] + K + 1, INF);
    priority_queue<tuple<ll, int, int>, vector<tuple<ll, int, int>>, greater<tuple<ll, int, int>>> pq;
    ds[s][0] = 0; pq.push({0, s, 0});
    auto relax = [&](int v, int k, ll nd) { if (nd < ds[v][k]) ds[v][k] = nd, pq.push({nd, v, k}); };
    while (!pq.empty()) {
        auto [d, u, k] = pq.top(); pq.pop();
        if (d > ds[u][k]) continue;
        for (auto [v, w] : adj[u]) {
            relax(v, k, d + w);                 // [改] 一般轉移
            if (k < K) relax(v, k + 1, d);      // [改] 特殊轉移
        }
    }
}
/* 常見的「額外資訊」(狀態數 × 轉移數 要 <= 約 1e7~1e8):
   | 題目條件                    | 狀態                  |
   | 最多 K 次免費/折扣/瞬移     | (v, 已用次數)         |
   | 收集鑰匙開門 (鑰匙 <= 10 種) | (x, y, 鑰匙 bitmask) |
   | 油箱容量 C、加油站          | (v, 剩餘油量)         |
   | 紅綠燈/週期 T 的時間限制    | (v, 時間 mod T)       |
   | 走奇數/偶數步、交替顏色邊   | (v, 奇偶 / 上一條邊顏色) |
   | 轉彎次數、不能連續同方向    | (x, y, 方向)          |
   | 是否經過特定點/是否已用技能 | (v, 0/1)              |
   邊權全 1 → BFS；只有 0/1 → 0-1 BFS；一般非負 → Dijkstra
   狀態之間若是 DAG (例如 k 只會增加且同層無環) 也可以直接 DP
*/
```

6.5 SPFA (Bellman-Ford)

用法: 可有負權邊、可判負環、差分約束。只在有負邊時用。
時間複雜度: 平均快，最差 $O(VE)$

```
// 可有負權邊的單源最短路 + 負環偵測。平均快，最差 O(VE)
const int MAXN = 100005;
const ll INF = 1e18;
int n, cnt_[MAXN];
ll dist[MAXN];
bool inq[MAXN];
vector<pair<int, ll>> g[MAXN];
bool spfa(int s) { // false = 從 s 走得到負環
    fill(dist, dist + n + 1, INF);
    fill(inq, inq + n + 1, false);
    fill(cnt_, cnt_ + n + 1, 0);
    queue<int> q;
    dist[s] = 0; q.push(s); inq[s] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop(); inq[u] = false;
        for (auto [v, w] : g[u]) if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (!inq[v]) {
                if (++cnt_[v] >= n) return false; // 入隊 >= n 次 → 負環
                q.push(v); inq[v] = true;
            }
        }
    }
    return true;
}
/* [變形]
- 判斷「整張圖」有沒有負環: 建超級源點 0 連到每個點權 0，從 0 跑 (n 要算進 0 → 用 n+1 判斷)
- 差分約束: 條件 x[v] - x[u] <= w → 邊 u→v 權 w。超級源點 0 → 每點權 0
  有負環 = 無解；否則 dist 就是一組解 (x[v]-x[u] >= w 改寫成 x[u]-x[v] <= -w)
- 最長路: 權重取負後跑最短路 (正環 = 最長路無限大)
*/
```

6.6 Floyd-Warshall 全點對最短路

用法: k 一定在最外層。附路徑還原、負環、遞移閉包。
時間複雜度: $O(V^3)$

```
// 全點對最短路 O(V^3)，V <= 500。可有負邊 (不可有負環)
const int MAXN = 505;
const ll INF = 1e18;
int n, nxt[MAXN][MAXN];
ll d[MAXN][MAXN];
void init() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) d[i][j] = (i == j ? 0 : INF), nxt[i][j] = j;
}
// 讀邊: d[u][v] = min(d[u][v], w); (重邊取 min！無向圖兩個方向都設)
void floyd() {
    for (int k = 1; k <= n; k++) // k (中繼點) 一定在最外層
        for (int i = 1; i <= n; i++) if (d[i][k] < INF)
            for (int j = 1; j <= n; j++) if (d[k][j] < INF && d[i][k] + d[k][j] < d[i][j]) {
                d[i][j] = d[i][k] + d[k][j]; // [改] 瓶頸: max(d[i][k], d[k][j]) 再取 min
                nxt[i][j] = nxt[i][k];       // 路徑還原
            }
}
vector<int> path(int i, int j) { // 需 d[i][j] < INF
    vector<int> p = {i};
    while (i != j) i = nxt[i][j], p.push_back(i);
    return p;
}
/* [變形]
- 有負環: 跑完後 d[i][i] < 0 的點在負環上
- 遞移閉包 (誰能到誰): bool 版 reach[i][j] |= reach[i][k] && reach[k][j]，可用 bitset 加速
- 最小環 (無向): 在 k 迴圈「更新前」先算 min(d[i][j] + w[i][k] + w[k][j])，i<j<k
*/
```

6.7 二分圖判定 (著色)

用法: 相鄰點塗不同色，衝突 = 有奇環。已處理不連通。
時間複雜度: $O(V+E)$

```cpp
// 二分圖判定（相鄰點塗不同色），已處理不連通。color: 0 未塗, 1 / -1
const int MAXN = 200005;
int n, color_[MAXN];
vector<int> g[MAXN];
bool is_bipartite() {
    fill(color_, color_ + n + 1, 0);
    for (int s = 1; s <= n; s++) {
        if (color_[s]) continue;
        queue<int> q; q.push(s); color_[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (!color_[v]) color_[v] = -color_[u], q.push(v);
                else if (color_[v] == color_[u]) return false; // 奇環
            }
        }
    }
    return true;
}
// 二分圖 <=> 沒有奇數環。分兩組/兩隊/兩種顏色的題目先想這個
// 每個連通塊可以「翻轉顏色」→ 各塊（黑數，白數）再做背包 DP 求最平均分組
```

6.8 割點、橋、點雙連通分量

用法: 一次 Tarjan 找出割點、橋與所有 v-BCC (孤立點自成一個)。用邊編號處理重邊。
時間複雜度: $O(V+E)$

```cpp
/* 無向圖。dfn[u]: DFS 拜訪順序；low[u]: u 的子樹「不經過父邊」能回到的最小 dfn
   割點: 非根 u 有子節點 v 使 low[v] >= dfn[u]；根需 >= 2 個 DFS 子樹
   點雙連通分量 (v-BCC): 內部任兩點有兩條點不重複路徑；割點可同時屬於多個 BCC */
const int MAXN = 200005;
int n, timer_, dfn[MAXN], low[MAXN];
bool is_cut[MAXN];
vector<pair<int, int>> g[MAXN]; // {鄰點, 邊編號}，邊編號用來處理重邊
vector<vector<int>> bcc;
vector<int> stk;
void add_edge(int u, int v, int id) { g[u].push_back({v, id});
g[v].push_back({u, id}); }
void tarjan(int u, int pe) { // pe = 走進 u 的邊編號
    dfn[u] = low[u] = ++timer_;
    stk.push_back(u);
    int child = 0;
    for (auto [v, id] : g[u]) {
        if (id == pe) continue;
        if (dfn[v]) { low[u] = min(low[u], dfn[v]); continue; } // 回邊
        child++;
        tarjan(v, id);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
            if (pe != -1) is_cut[u] = true;
            vector<int> comp = {u}; // u 留在 stack 上（可能還屬於別的 BCC）
            while (true) {
                int w = stk.back(); stk.pop_back();
                comp.push_back(w);
                if (w == v) break;
            }
            bcc.push_back(comp);
        }
    }
    if (pe == -1 && child >= 2) is_cut[u] = true;
}
void solve() {
    timer_ = 0; bcc.clear();
    fill(dfn, dfn + n + 1, 0); fill(is_cut, is_cut + n + 1, false);
    for (int i = 1; i <= n; i++) if (!dfn[i]) {
        stk.clear();
        tarjan(i, -1);
        if (g[i].empty()) bcc.push_back({i}); // 孤立點自成一個 BCC
    }
}
```

6.9 匈牙利演算法 (二分圖最大匹配)

用法: 左右兩側，match_r[v] 記錄右邊 v 配到誰。附二分圖重要定理。
時間複雜度: $O(VE)$

```cpp
// 二分圖最大匹配 (Kuhn)，O(V*E)。左邊 1..nl，右邊 1..nr
const int MAXN = 1005;
int nl, nr, match_r[MAXN]; // match_r[v] = 右邊 v 配到的左邊點 (0 = 沒配)
bool vis[MAXN];
vector<int> g[MAXN];       // g[左 u] = 可配的右邊點
bool try_match(int u) {
    for (int v : g[u]) {
        if (vis[v]) continue;
        vis[v] = true;
        if (!match_r[v] || try_match(match_r[v])) { match_r[v] = u; return
        true; } // 搶位子
    }
    return false;
}
int max_matching() {
    fill(match_r, match_r + nr + 1, 0);
    int res = 0;
    for (int u = 1; u <= nl; u++) {
        fill(vis, vis + nr + 1, false);
        if (try_match(u)) res++;
    }
    return res;
}
/* 二分圖定理 (重要!):
   最小點覆蓋 = 最大匹配              (König)
   最大獨立集 = 總點數 - 最大匹配
   DAG 最小路徑覆蓋（路徑不相交）= 點數 - 最大匹配（每點拆成 左 u、右 u，邊 u→v
   變 左 u-右 v）
   點數很大 (1e5) 改用 Dinic，複雜度 O(E sqrt V) */
```

6.10 KM 演算法

用法: 帶權二分圖最大權完美匹配，左右各 N 點 (不足補虛點)。
時間複雜度: $O(N^3)$

```cpp
// 帶權二分圖「最大權完美匹配」，左右各 n 點，O(n^3)。1-based
// 點數不同或沒有邊: 補虛擬點/權重 0 的邊（不能選的邊設 -INF 級大負數）
// 要「最小權」: 權重全部取負，答案再取負
const int MAXN = 505;
const ll INF = 1e18;
int n, match_y[MAXN], pre_[MAXN];
ll w[MAXN][MAXN], lx[MAXN], ly[MAXN], slack[MAXN];
bool vis_y[MAXN];
void km_bfs(int root) {
    fill(slack, slack + n + 1, INF);
    fill(vis_y, vis_y + n + 1, false);
    fill(pre_, pre_ + n + 1, 0);
    int y = 0;
    match_y[0] = root;
    while (true) {
        int x = match_y[y], ny = 0;
        ll delta = INF;
        vis_y[y] = true;
        for (int i = 1; i <= n; i++) if (!vis_y[i]) {
            ll d = lx[x] + ly[i] - w[x][i];
            if (d < slack[i]) slack[i] = d, pre_[i] = y;
            if (slack[i] < delta) delta = slack[i], ny = i;
        }
        for (int i = 0; i <= n; i++) {
            if (vis_y[i]) lx[match_y[i]] -= delta, ly[i] += delta;
            else slack[i] -= delta;
        }
        y = ny;
        if (match_y[y] == 0) break; // 找到沒配對的右邊點
    }
    while (y) match_y[y] = match_y[pre_[y]], y = pre_[y]; // 沿路翻轉
}
ll km() { // 先填好 n 與 w[1..n][1..n]
    fill(match_y, match_y + n + 1, 0);
    fill(ly, ly + n + 1, 0);
    for (int i = 1; i <= n; i++) lx[i] = *max_element(w[i] + 1, w[i] + n +
    1);
    for (int i = 1; i <= n; i++) km_bfs(i);
    ll res = 0;
    for (int i = 1; i <= n; i++) res += w[match_y[i]][i]; // 右 i 配到左
    match_y[i]
    return res;
}
```

6.11 Dinic 最大流

用法: 反向邊 $=$ e^1。附最小割與常見建模。多筆測資呼叫 init(n)。
時間複雜度: $O(V^2E)$，二分圖 $O(E\sqrt{V})$

```cpp
// 最大流 Dinic。O(V^2 E)，二分圖 O(E sqrt V)。邊 e 的反向邊是 e ^ 1
const int MAXN = 5005, MAXE = 200005; // MAXE = 2 * 邊數
const ll INF = 1e18;
int n, S, T, ecnt, to[MAXE], lvl[MAXN], it_[MAXN];
ll cap[MAXE];
vector<int> g[MAXN]; // g[u] = 從 u 出發的邊編號
void init(int _n) { n = _n; ecnt = 0; for (int i = 0; i <= n; i++)
g[i].clear(); }
void add_edge(int u, int v, ll c) { // 無向邊: 第二行的 0 改成 c
    to[ecnt] = v; cap[ecnt] = c; g[u].push_back(ecnt++);
    to[ecnt] = u; cap[ecnt] = 0; g[v].push_back(ecnt++);
}
bool bfs() {
    fill(lvl, lvl + n + 1, -1);
    queue<int> q; q.push(S); lvl[S] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int e : g[u]) if (cap[e] > 0 && lvl[to[e]] == -1) lvl[to[e]] =
        lvl[u] + 1, q.push(to[e]);
    }
    return lvl[T] != -1;
}
ll dfs(int u, ll f) {
    if (u == T) return f;
    for (int &i = it_[u]; i < (int)g[u].size(); i++) { // 當前弧優化: i 是參考
        int e = g[u][i], v = to[e];
        if (cap[e] > 0 && lvl[v] == lvl[u] + 1) {
            ll d = dfs(v, min(f, cap[e]));
            if (d > 0) { cap[e] -= d; cap[e ^ 1] += d; return d; }
        }
    }
    return 0;
}
ll max_flow(int s, int t) {
    S = s; T = t;
    ll flow = 0;
    while (bfs()) {
        fill(it_, it_ + n + 1, 0);
        while (ll f = dfs(S, INF)) flow += f;
    }
    return flow;
}
/* 建模技巧:
- 最小割 = 最大流。割邊: 跑完後從 S 沿 cap>0 走得到的點集合 A，A→ 非 A 的原始邊
- 二分圖匹配: S→ 左 (1)、左 → 右 (1)、右 →T(1)；每點最多配 k 個 → 對應邊容量 k
- 點有容量: 拆點 v_in → v_out 容量 = 點容量
- 多源多匯: 超級源點連所有源點（容量 INF）
- 最大權閉合子圖（選 A 必須選 B): S→ 正權點 (w)、負權點 →T(-w)、A→B(INF)；答案
= 正權和 - 最大流
*/
```

7 樹

7.1 Diameter 直徑

用法: 兩次 BFS 找最遠點，支援邊權 (≥ 0)。
時間複雜度: $O(N)$

```
// 樹直徑: 從任意點找最遠點 a，再從 a 找最遠點 b，dist(a,b) 即直徑。邊權須 >= 0
const int MAXN = 200005;
int n, par_[MAXN];
ll d[MAXN];
vector<pair<int, ll>> g[MAXN]; // 無權樹: w = 1
int farthest(int s) { // BFS 版 (樹上路徑唯一，不必 Dijkstra)，避免遞迴太深
    fill(d, d + n + 1, -1);
    queue<int> q; q.push(s); d[s] = 0; par_[s] = 0;
    int best = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (auto [v, w] : g[u]) if (d[v] == -1) d[v] = d[u] + w, par_[v] =
        u, q.push(v);
    }
    return best;
}
ll diameter(int &a, int &b) {
    a = farthest(1); b = farthest(a);
    return d[b]; // 路徑: 從 b 沿 par_ 走回 a
}
/* [應用]
- 每個點的最遠距離 = max(dist(v,a), dist(v,b))  (各跑一次 BFS)
- 樹的中心 (最小化最遠距離): 直徑路徑的中點；從一點廣播到全樹的最短時間 =
ceil(直徑/2)
*/
```

7.2 Traversal 遍歷

用法: 陣列存二元樹的前/中/後序、層序；由前序 + 中序求後序。
時間複雜度: $O(N)$ (重建 $O(N^2)$)

```
// 二元樹用陣列存: lc[u], rc[u] (0 = 沒有子節點)，root 通常為 1
const int MAXN = 200005;
int lc[MAXN], rc[MAXN];
vector<int> pre_order, in_order, post_order;
void dfs(int u) {
    if (!u) return;
    pre_order.push_back(u);   // 前序: 根 左 右
    dfs(lc[u]);
    in_order.push_back(u);    // 中序: 左 根 右 (BST 的中序 = 排序好的)
    dfs(rc[u]);
    post_order.push_back(u);  // 後序: 左 右 根
}
vector<int> level_order(int root) { // 層序 (BFS)
    vector<int> res; queue<int> q; q.push(root);
    while (!q.empty()) {
        int u = q.front(); q.pop(); res.push_back(u);
        if (lc[u]) q.push(lc[u]);
        if (rc[u]) q.push(rc[u]);
    }
    return res;
}
// 給前序 + 中序，輸出後序 (字元版)。O(N^2)，N 大時用 map 記 in 的位置
string pre_in_to_post(const string &pre, const string &in) {
    if (pre.empty()) return "";
    char root = pre[0];
    int k = in.find(root);    // 左子樹大小
    return pre_in_to_post(pre.substr(1, k), in.substr(0, k))
         + pre_in_to_post(pre.substr(k + 1), in.substr(k + 1)) + root;
}
// 給後序 + 中序 → 根在後序最後一個，其餘同理
// 一般樹 (不只二元): vector<int> ch[u] 存小孩，DFS 遞迴每個小孩即可
```

7.3 LCA 最近共同祖先 (倍增)

用法: BFS 建表 (不怕鏈狀樹)。附第 k 個祖先、兩點距離、路徑最大邊權。
時間複雜度: 預處理 $O(N \log N)$，查詢 $O(\log N)$

```
// 倍增 LCA: 預處理 O(N log N)、查詢 O(log N)。同時求路徑上的最大邊權
const int MAXN = 200005, LOG = 18; // 2^LOG > N
int n, up[LOG][MAXN], dep[MAXN];
ll mx[LOG][MAXN], distr[MAXN];     // mx[j][v]: v 往上 2^j 條邊中的最大邊權；
distr: 到根距離
vector<pair<int, ll>> g[MAXN];
void build(int root) { // BFS 版，鏈狀樹也不會 stack overflow
    vector<int> order; order.reserve(n);
    fill(dep, dep + n + 1, -1);
    queue<int> q; q.push(root);
    dep[root] = 0; up[0][root] = root; mx[0][root] = 0; distr[root] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (auto [v, w] : g[u]) if (dep[v] == -1) {
            dep[v] = dep[u] + 1; up[0][v] = u; mx[0][v] = w; distr[v] =
            distr[u] + w;
            q.push(v);
        }
    }
    for (int j = 1; j < LOG; j++)
        for (int v : order) {
            up[j][v] = up[j - 1][up[j - 1][v]];
            mx[j][v] = max(mx[j - 1][v], mx[j - 1][up[j - 1][v]]); // [改]
            min / sum
        }
}
int kth_ancestor(int v, int k) { for (int j = 0; j < LOG; j++) if (k >> j &
1) v = up[j][v]; return v; }
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    u = kth_ancestor(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; j--)
        if (up[j][u] != up[j][v]) u = up[j][u], v = up[j][v];
    return up[0][u];
}
ll path_max(int u, int v) { // u-v 路徑上的最大邊權
    ll res = 0;
    if (dep[u] < dep[v]) swap(u, v);
    for (int j = LOG - 1; j >= 0; j--)
        if (dep[u] - (1 << j) >= dep[v]) res = max(res, mx[j][u]), u =
        up[j][u];
    if (u == v) return res;
    for (int j = LOG - 1; j >= 0; j--)
        if (up[j][u] != up[j][v]) {
            res = max({res, mx[j][u], mx[j][v]});
            u = up[j][u]; v = up[j][v];
        }
    return max({res, mx[0][u], mx[0][v]});
}
// 兩點距離 = distr[u] + distr[v] - 2 * distr[lca(u, v)]  (邊數: 用 dep)
// u 是否在 v 的子樹內: lca(u, v) == v
```

7.4 Kruskal 最小生成樹

用法: 邊排序後用 DSU 判環。需要 DisjointSet.cpp。
時間複雜度: $O(E \log E)$

```
// 最小生成樹 O(E log E)。需要 DisjointSet.cpp 的 init / unite
int n;
vector<tuple<ll, int, int>> edges;    // {w, u, v}，tuple 會先比 w
vector<pair<int, ll>> mst_adj[200005]; // MST 本身 (之後要在樹上做事用)
ll kruskal() { // 不連通回傳 -1
    sort(edges.begin(), edges.end()); // [改] 最大生成樹:
    sort(edges.rbegin(), edges.rend())
    init(n);
    for (int i = 0; i <= n; i++) mst_adj[i].clear();
    ll total = 0; int used = 0;
    for (auto [w, u, v] : edges)
        if (unite(u, v)) {
            total += w; used++;
            mst_adj[u].push_back({v, w}); mst_adj[v].push_back({u, w});
        }
    return used == n - 1 ? total : -1;
}
/* [變形]
- 瓶頸: 「u 到 v 路徑上最大邊最小」= MST 上 u-v 路徑的最大邊 (LCA path_max)
- 必須包含某些邊: 先把那些邊 unite 並加總，再跑 Kruskal
- 最小生成「森林」/ 恰好 k 個連通塊: 做到剩 k 塊 (used == n - k) 就停
- 稠密圖 (V <= 5000, E ~ V^2): 用 O(V^2) Prim 比較快
*/
```

7.5 嚴格次小生成樹

用法: MST + 倍增維護路徑上最大與嚴格次大邊。需要 DisjointSet.cpp。
時間複雜度: $O(E \log V)$

```
/* 嚴格次小生成樹 O(E log V)。需要 DisjointSet.cpp
   想法: 先求 MST；對每條「非樹邊」(u,v,w)，加進去會形成環，
   拿掉環上 (= 樹上 u-v 路徑) 最大且 < w 的邊 → 候選 = MST - 那條邊 + w
   所以要維護路徑上的「最大值」與「嚴格次大值」 */
const int LOG = 18; // MAXN 沿用 DisjointSet.cpp 的 (2^LOG > N)
const ll INF = 1e18;
int n, up[LOG][MAXN], dep[MAXN];
ll m1[LOG][MAXN], m2[LOG][MAXN]; // 往上 2^j 條邊的最大、嚴格次大
vector<pair<int, ll>> t[MAXN];
void merge_(ll &a1, ll &a2, ll b1, ll b2) { // 把 (b1,b2) 併進 (a1,a2)
    for (ll x : {b1, b2}) {
        if (x > a1) a2 = a1, a1 = x;
        else if (x < a1 && x > a2) a2 = x;
    }
}
ll second_mst(vector<tuple<ll, int, int>> edges) { // 回傳 -1 表示不存在
    sort(edges.begin(), edges.end());
    init(n);
    vector<bool> in_mst(edges.size(), false);
    ll mst = 0; int used = 0;
    for (int i = 0; i < (int)edges.size(); i++) {
        auto [w, u, v] = edges[i];
        if (unite(u, v)) in_mst[i] = true, mst += w, used++,
        t[u].push_back({v, w}), t[v].push_back({u, w});
    }
    if (used != n - 1) return -1;
    vector<int> order; queue<int> q; q.push(1);          // BFS 建倍增表
    fill(dep, dep + n + 1, -1); dep[1] = 0; up[0][1] = 1; m1[0][1] =
    m2[0][1] = -INF;
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (auto [v, w] : t[u]) if (dep[v] == -1)
            dep[v] = dep[u] + 1, up[0][v] = u, m1[0][v] = w, m2[0][v] =
            -INF, q.push(v);
    }
    for (int j = 1; j < LOG; j++) for (int v : order) {
        int mid = up[j - 1][v];
        up[j][v] = up[j - 1][mid];
        m1[j][v] = m1[j - 1][v]; m2[j][v] = m2[j - 1][v];
        merge_(m1[j][v], m2[j][v], m1[j - 1][mid], m2[j - 1][mid]);
    }
    ll best = INF;
    for (int i = 0; i < (int)edges.size(); i++) {
        if (in_mst[i]) continue;
        auto [w, u, v] = edges[i];
        ll a1 = -INF, a2 = -INF;
        if (dep[u] < dep[v]) swap(u, v);
        for (int j = LOG - 1; j >= 0; j--)
            if (dep[u] - (1 << j) >= dep[v]) merge_(a1, a2, m1[j][u],
            m2[j][u]), u = up[j][u];
        if (u != v) {
            for (int j = LOG - 1; j >= 0; j--) if (up[j][u] != up[j][v]) {
```

```
                merge_(a1, a2, m1[j][u], m2[j][u]); merge_(a1, a2, m1[j][v],
                m2[j][v]);
                u = up[j][u]; v = up[j][v];
            }
            merge_(a1, a2, m1[0][u], m2[0][u]); merge_(a1, a2, m1[0][v],
            m2[0][v]);
        }
        if (a1 < w) best = min(best, mst - a1 + w);          // 不要求嚴格:
        a1 <= w 即可
        else if (a2 > -INF) best = min(best, mst - a2 + w);
    }
    return best == INF ? -1 : best;
}
// 多筆測資: 記得 t[i].clear()
```

8 幾何

A. 點與直線

點 (x_0, y_0) 到直線 $ax + by + c = 0$ 的距離 $\frac{|ax_0+by_0+c|}{\sqrt{a^2+b^2}}$；兩平行線 $ax + by + c_1 = 0$、$ax + by + c_2 = 0$ 距離 $\frac{|c_1-c_2|}{\sqrt{a^2+b^2}}$。

B. 三角形

頂點 $A(x_1, y_1), B(x_2, y_2), C(x_3, y_3)$，對邊長 a, b, c，面積 Δ。

$$\Delta = \frac{1}{2}\left|\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}\right|, \quad \Delta = \sqrt{s(s-a)(s-b)(s-c)},\ s = \frac{a+b+c}{2}$$

重心 $G = \frac{A+B+C}{3}$；內心 $I = \frac{aA+bB+cC}{a+b+c}$；外心 O 見 Circle.cpp；垂心 $H = 3G - 2O$ (重心、外心、垂心共線)。

$$O_x = \frac{1}{4\Delta}\begin{vmatrix} x_1^2+y_1^2 & y_1 & 1 \\ x_2^2+y_2^2 & y_2 & 1 \\ x_3^2+y_3^2 & y_3 & 1 \end{vmatrix}, \quad O_y = \frac{1}{4\Delta}\begin{vmatrix} x_1 & x_1^2+y_1^2 & 1 \\ x_2 & x_2^2+y_2^2 & 1 \\ x_3 & x_3^2+y_3^2 & 1 \end{vmatrix}$$

餘弦定理 $c^2 = a^2 + b^2 - 2ab\cos C$；正弦定理 $\frac{a}{\sin A} = 2R$；內切圓半徑 $r = \frac{\Delta}{s}$；外接圓 $R = \frac{abc}{4\Delta}$。

C. 四邊形

凸四邊形邊長 a, b, c, d，$s = \frac{a+b+c+d}{2}$，已知 $\angle A + \angle C$：面積 $= \sqrt{(s-a)(s-b)(s-c)(s-d) - abcd\cos^2\frac{A+C}{2}}$。圓內接四邊形時第二項為 0。只知道一個角：用餘弦定理求對角線，再用海龍公式算兩個三角形。

8.1 點與向量基礎

用法: 點用 `pair<ld,ld>` 表示，附 `+ - * /`、內積 `dot`、外積 `cross`、`sign` (處理誤差)。座標全為整數時把 `ld` 改 `ll`。
時間複雜度: $O(1)$

```
// 點/向量用 pair 表示 (.first = x, .second = y)。座標全是整數時把 ld 改成 ll，
完全沒有誤差
typedef long double ld;
typedef pair<ld, ld> P;
const ld EPS = 1e-9, PI = acosl(-1);
int sign(ld x) { return fabsl(x) < EPS ? 0 : (x < 0 ? -1 : 1); } // 浮點數比
較一律用 sign
P operator+(P a, P b) { return {a.first + b.first, a.second + b.second}; }
P operator-(P a, P b) { return {a.first - b.first, a.second - b.second}; }
P operator*(P a, ld k) { return {a.first * k, a.second * k}; }
P operator/(P a, ld k) { return {a.first / k, a.second / k}; }
ld dot(P a, P b) { return a.first * b.first + a.second * b.second; }   // > 0
銳角, = 0 垂直, < 0 鈍角
ld cross(P a, P b) { return a.first * b.second - a.second * b.first; } // >
0: b 在 a 的逆時針側 (左邊)
ld cross(P o, P a, P b) { return cross(a - o, b - o); }                //
o→a→b: > 0 左轉, < 0 右轉, = 0 共線
ld len(P a) { return sqrtl(dot(a, a)); }
bool same(P a, P b) { return sign(a.first - b.first) == 0 && sign(a.second -
b.second) == 0; }
P rotate(P a, ld th) { return {a.first * cosl(th) - a.second * sinl(th),
a.first * sinl(th) + a.second * cosl(th)}; } // 逆時針
ld angle(P a) { return atan2l(a.second, a.first); } // (-PI, PI]
// 極角排序: sort(v.begin(), v.end(), [](P a, P b){ return angle(a) <
angle(b); });
```

8.2 線段相交、直線交點

用法: 外積跨立實驗，可處理端點接觸與共線重疊；矩形交集面積公式。
時間複雜度: $O(1)$

```
// 需要 point.cpp
bool on_segment(P p, P a, P b) { // p 是否在線段 ab 上 (含端點)
    return sign(cross(a, b, p)) == 0 && sign(dot(a - p, b - p)) <= 0;
}
// 線段 ab 與 cd 是否相交 (含端點接觸、共線重疊)
bool seg_intersect(P a, P b, P c, P d) {
    int d1 = sign(cross(a, b, c)), d2 = sign(cross(a, b, d));
    int d3 = sign(cross(c, d, a)), d4 = sign(cross(c, d, b));
    if (d1 * d2 < 0 && d3 * d4 < 0) return true; // 嚴格交叉 (跨立實驗)
    return on_segment(c, a, b) || on_segment(d, a, b) || on_segment(a, c, d)
    || on_segment(b, c, d);
}
// 直線 ab 與直線 cd 的交點 (先確認不平行: sign(cross(b-a, d-c)) != 0)
P line_intersection(P a, P b, P c, P d) {
    ld t = cross(c - a, d - c) / cross(b - a, d - c);
    return a + (b - a) * t;
}
/* 軸對齊矩形交集 (左下 (x1,y1) 右上 (x2,y2))，與 (x3,y3)-(x4,y4)):
   W = max(0, min(x2,x4) - max(x1,x3)), H = max(0, min(y2,y4) - max(y1,y3))，
   面積 = W * H
   (只問「有沒有碰到」: W >= 0 && H >= 0) */
```

8.3 點與直線/線段

用法: 點到直線、點到線段距離；投影點、鏡射點。
時間複雜度: $O(1)$

```
// 需要 point.cpp
ld dist_to_line(P p, P a, P b) { // 點到「直線」ab: 平行四邊形面積 / 底
    return fabsl(cross(a, b, p)) / len(b - a);
}
ld dist_to_segment(P p, P a, P b) { // 點到「線段」ab
    if (same(a, b)) return len(p - a);
    if (sign(dot(b - a, p - a)) < 0) return len(p - a); // 投影落在 a 外側
    if (sign(dot(a - b, p - b)) < 0) return len(p - b); // 投影落在 b 外側
    return dist_to_line(p, a, b);
}
P projection(P p, P a, P b) { // p 在直線 ab 上的投影點 (垂足)
    return a + (b - a) * (dot(p - a, b - a) / dot(b - a, b - a));
}
P reflection(P p, P a, P b) { return projection(p, a, b) * 2 - p; } // 對直線
ab 的鏡射
// 兩線段距離 (不相交時) = 四個「端點到另一線段」距離取 min
```

8.4 多邊形面積、點在多邊形內

用法: 鞋帶公式 (凹多邊形可)；射線法判斷內/外/邊上；Pick 定理。
時間複雜度: $O(N)$

```
// 需要 point.cpp。多邊形頂點依序 (順/逆時針皆可) 存在 vector<P> poly
ld polygon_area(const vector<P> &poly) { // 鞋帶公式，凹多邊形也可
    ld area = 0; int n = poly.size();
    for (int i = 0; i < n; i++) area += cross(poly[i], poly[(i + 1) % n]);
    return fabsl(area) / 2; // 不取絕對值時: > 0 代表逆時針
}
// 點是否在多邊形內: 1 內部, 0 外部, -1 在邊上 (射線法)
int in_polygon(P p, const vector<P> &poly) {
    int n = poly.size(); bool in = false;
    for (int i = 0; i < n; i++) {
        P a = poly[i], b = poly[(i + 1) % n];
        if (on_segment(p, a, b)) return -1;                  // 需要
        Intersection.cpp
        if ((a.second > p.second) != (b.second > p.second) &&
            p.first < a.first + (b.first - a.first) * (p.second - a.second)
            / (b.second - a.second))
            in = !in;
    }
    return in;
}
/* Pick 定理 (頂點都是格子點): 面積 A = I + B/2 - 1
   I = 內部格點數，B = 邊上格點數 = Σ gcd(|dx|, |dy|)  → I = A - B/2 + 1 */
```

8.5 凸包 Convex Hull、旋轉卡尺

用法: Andrew 單調鏈，回傳逆時針凸包；旋轉卡尺求最遠點對。
時間複雜度: $O(N \log N)$

```
// 需要 point.cpp。Andrew 單調鏈，O(N log N)，回傳逆時針凸包 (不含共線點)
vector<P> convex_hull(vector<P> pts) {
    sort(pts.begin(), pts.end()); // pair 預設先比 x 再比 y
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n <= 2) return pts;
    vector<P> h(2 * n);
    int m = 0;
    for (int i = 0; i < n; i++) { // 下凸包 (左 → 右)
        while (m >= 2 && sign(cross(h[m - 2], h[m - 1], pts[i])) <= 0) m--;
        // [改] 保留共線點: < 0
        h[m++] = pts[i];
    }
    for (int i = n - 2, t = m + 1; i >= 0; i--) { // 上凸包 (右 → 左)
        while (m >= t && sign(cross(h[m - 2], h[m - 1], pts[i])) <= 0) m--;
        h[m++] = pts[i];
    }
    h.resize(m - 1); // 起點被加了兩次
    return h;
}
// 旋轉卡尺: 凸包直徑 (最遠點對距離的平方)，h 為逆時針凸包
ld farthest_pair(const vector<P> &h) {
    int n = h.size();
    if (n == 1) return 0;
    if (n == 2) return dot(h[0] - h[1], h[0] - h[1]);
    ld best = 0;
    for (int i = 0, j = 1; i < n; i++) {
        while (cross(h[i], h[(i + 1) % n], h[(j + 1) % n]) > cross(h[i], h[(i
        + 1) % n], h[j])) j = (j + 1) % n;
        best = max({best, dot(h[i] - h[j], h[i] - h[j]), dot(h[(i + 1) % n]
        - h[j], h[(i + 1) % n] - h[j])});
    }
    return best;
}
// 凸包周長 = Σ len(h[i+1] - h[i])；面積用 polygon_area(h)
```

8.6 圓

用法： 三點求外接圓、直線與圓的交點、兩圓關係。
時間複雜度： $O(1)$

```
// 需要 point.cpp。圓用 {圓心 P, 半徑 ld} 表示
// 三點決定的圓（三角形外接圓）。三點共線時半徑回傳 -1
pair<P, ld> circumcircle(P a, P b, P c) {
    P ab = b - a, ac = c - a;
    ld d = 2 * cross(ab, ac);
    if (sign(d) == 0) return {{0, 0}, -1};
    ld b2 = dot(ab, ab), c2 = dot(ac, ac);
    P o = a + P{(ac.second * b2 - ab.second * c2) / d, (ab.first * c2 - ac.first * b2) / d};
    return {o, len(o - a)};
}
// 直線 ab 與圓 (o, r) 的交點 (0~2 個)
vector<P> circle_line(P o, ld r, P a, P b) {
    P h = projection(o, a, b);                  // 需要 PointLine.cpp
    ld d2 = r * r - dot(h - o, h - o);
    if (sign(d2) < 0) return {};
    P dir = (b - a) / len(b - a);
    ld s = sqrtl(max((ld)0, d2));
    if (sign(s) == 0) return {h};
    return {h - dir * s, h + dir * s};
}
// 兩圓關係 (d = 圓心距): d > r1+r2 相離；d == r1+r2 外切；|r1-r2| < d < r1+r2 相交；
// d == |r1-r2| 內切；d < |r1-r2| 內含
```

9 DP

DP 四步驟：1. 定義狀態：寫清楚 dp[i][j] 代表什麼 (**題目多一個條件 = 多一維**)。2. 轉移：「最後一步」是什麼？由哪些較小的狀態來？3. 初始值：dp[0]、不可能的狀態設 $\pm$INF。4. 計算順序：確保用到的狀態都已算好 (通常由小到大)。答案在哪一格？
常見狀態：前 i 個 (線性)、區間 $[l, r]$ (區間)、拓樸序 (DAG)。

9.1 背包問題

用法： 0/1 (含回溯)、**二維費用 (兩種資源同時限制，例: 時間 + 雞蛋)**、完全、多重 (二進位拆分)、分組；子集合和 bitset、方法數、容量很大時的變形。
時間複雜度： $O(NW)$

```
// n 個物品 (1-based)，重量 w[i]、價值 v[i]，容量 W
const int MAXN = 1005, MAXW = 100005;
int n, W, w[MAXN], v[MAXN];
ll dp[MAXW];
ll dp2[105][10005]; // 二維版很吃記憶體 (N*W*8 bytes)，只在要回溯時用

// 1. 0/1 背包 二維（要回溯選了哪些時用）。dp2[i][j] = 前 i 個物品、容量 j 的最大價值
void knap01_2d() {
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= W; j++) {
            dp2[i][j] = dp2[i - 1][j];
            // 不拿 i
            if (j >= w[i]) dp2[i][j] = max(dp2[i][j], dp2[i - 1][j - w[i]] + v[i]); // 拿 i
        }
    vector<int> chosen; // 回溯：和上一列一樣就是沒拿
    for (int i = n, j = W; i >= 1; i--)
        if (dp2[i][j] != dp2[i - 1][j]) chosen.push_back(i), j -= w[i];
}
// 2. 0/1 背包 一維（滾動陣列）：j 必須「由大到小」，否則同一物品會被拿多次
void knap01() {
    fill(dp, dp + W + 1, 0);        // [改] 恰好裝滿：dp[0]=0，其餘 -INF
    for (int i = 1; i <= n; i++)
        for (int j = W; j >= w[i]; j--) dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
}
// 2b. 二維費用 0/1 背包：每個物品同時消耗兩種資源（例：時間 t[i]、雞蛋 e[i]），上限 T、E
//     dp[j][k] = 時間用 <= j、雞蛋用 <= k 時的最大價值。和 0/1 一樣，兩層容量都「由大到小」
const int MAXT = 505, MAXE = 505;
int T, E, t[MAXN], e[MAXN];
ll dp2d[MAXT][MAXE];
void knap01_2cost() {
    for (int j = 0; j <= T; j++) fill(dp2d[j], dp2d[j] + E + 1, 0);
    for (int i = 1; i <= n; i++)
        for (int j = T; j >= t[i]; j--)          // 時間由大到小
            for (int k = E; k >= e[i]; k--)      // 雞蛋由大到小
                dp2d[j][k] = max(dp2d[j][k], dp2d[j - t[i]][k - e[i]] + v[i]);
    // 答案 dp2d[T][E]。[改] 每個物品可無限拿：兩層都改成由小到大
    // [改] 求「最多能做幾個」：v[i] = 1
}
// 3. 完全背包（每種無限個）：j「由小到大」
void knap_unbounded() {
    fill(dp, dp + W + 1, 0);
    for (int i = 1; i <= n; i++)
        for (int j = w[i]; j <= W; j++) dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
}
// 4. 多重背包（第 i 種有 c[i] 個）：二進位拆分成 1,2,4,..., 剩餘 → 變 0/1 背包，O(W Σ log c)
int c[MAXN];
void knap_bounded() {
    fill(dp, dp + W + 1, 0);
    for (int i = 1; i <= n; i++)
        for (int k = 1, left = c[i]; left > 0; k <<= 1) {
            int take = min(k, left); left -= take;
            int ww = w[i] * take; ll vv = (ll)v[i] * take;
            for (int j = W; j >= ww; j--) dp[j] = max(dp[j], dp[j - ww] + vv);
        }
}
// 5. 分組背包（每組最多選一個）：容量迴圈在外（由大到小），組內物品在內
vector<vector<pair<int, int>>> groups; // groups[g] = {(重量, 價值)}
void knap_group() {
    fill(dp, dp + W + 1, 0);
    for (auto &grp : groups)
        for (int j = W; j >= 0; j--)
            for (auto [gw, gv] : grp) if (j >= gw) dp[j] = max(dp[j], dp[j - gw] + gv);
}
/* 6. 子集合和 / 能否剛好湊出 / 分成兩堆等重：bitset<MAXW> b; b[0]=1; 每個 x: b |= b << x;
      能湊出 j <=> b[j]。等重兩堆：總和偶數且 b[sum/2]
   7. 方法數：dp[0]=1，max 改成 +=
   8. 容量很大（1e9）但價值小：改成 dp[價值] = 最小重量，最後找 dp[val] <= W 的最大 val
   9. 多人各自有容量 W_i、共用同一批商品：跑一次 dp，答案 Σ dp[W_i]（dp 初始全 0 時已是單調）*/
```

9.2 分錢幣問題

用法： 完全背包的變形：組合數 (外層硬幣)、排列數 (外層金額)、最少硬幣數。
時間複雜度： $O(\text{硬幣數} \times K)$

```
// 湊出金額 k（硬幣無限使用）。方法數成長很快 → 取模或 long long
const int MAXK = 1000005;
const ll MOD = 1e9 + 7, INF = 1e18;
ll dp[MAXK];
// 1. 組合數（2+3 和 3+2 算同一種）：外層「硬幣」、內層「金額」
ll count_combination(const vector<int> &coins, int k) {
    fill(dp, dp + k + 1, 0); dp[0] = 1;
    for (int c : coins)
        for (int i = c; i <= k; i++) dp[i] = (dp[i] + dp[i - c]) % MOD;
    return dp[k];
}
// 2. 排列數（順序不同算不同）：外層「金額」、內層「硬幣」
ll count_permutation(const vector<int> &coins, int k) {
    fill(dp, dp + k + 1, 0); dp[0] = 1;
    for (int i = 1; i <= k; i++)
        for (int c : coins) if (i >= c) dp[i] = (dp[i] + dp[i - c]) % MOD;
    return dp[k];
}
// 3. 最少硬幣數（湊不出回傳 -1）
ll min_coins(const vector<int> &coins, int k) {
    fill(dp, dp + k + 1, INF); dp[0] = 0;
    for (int c : coins)
        for (int i = c; i <= k; i++) if (dp[i - c] != INF) dp[i] = min(dp[i], dp[i - c] + 1);
    return dp[k] == INF ? -1 : dp[k];
}
// 多筆詢問同一組硬幣：在 main 開頭對最大金額建一次表，每筆直接查 dp[k]
// 每種硬幣只能用一次 → 內層改成由大到小（變 0/1 背包）
```

9.3 LCS 最長共同子序列、編輯距離

用法： 二維 DP + 回溯出字串。
時間複雜度： $O(NM)$

```
// dp[i][j] = a 前 i 個字元、b 前 j 個字元 的答案。index 0 = 空字串
const int MAXN = 3005;   // 5000x5000 int = 100MB 太大，注意記憶體
int dp[MAXN][MAXN];
string lcs(const string &a, const string &b) { // 最長共同子序列（回傳字串）
    int n = a.size(), m = b.size();
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0) dp[i][j] = 0;
            else if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    string res;
    for (int i = n, j = m; i > 0 && j > 0;) { // 回溯
        if (a[i - 1] == b[j - 1]) res += a[i - 1], i--, j--;
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }
    reverse(res.begin(), res.end());
    return res; // 長度 = dp[n][m]
}
int edit_distance(const string &a, const string &b) { // 插入/刪除/取代 各 1 次
    int n = a.size(), m = b.size();
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0) { dp[i][j] = i + j; continue; }  // 全部插入/刪除
            dp[i][j] = min({dp[i - 1][j] + 1,                          // 刪 a[i-1]
                            dp[i][j - 1] + 1,                          // 插 b[j-1]
                            dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])}); // 取代/相同
        }
    return dp[n][m]; // 輸出過程：從 (n,m) 往回看是哪一項轉移來的
}
/* - 最長迴文子序列 = lcs(s, reverse(s)).size()；最少插入幾個字變迴文 = n - 它
   - 最長共同「子字串」(要連續)：相同時 dp = dp[i-1][j-1]+1，不同時 dp = 0，答案取全部最大
   - 只有插入/刪除（沒有取代）的距離 = n + m - 2 * LCS
   - 空間不夠：只保留兩列 (dp[2][MAXN])，用 i & 1 */
```

9.4 LIS 最長遞增子序列

用法： 嚴格用 `lower_bound`、非嚴格用 `upper_bound`。附輸出真正的 LIS 與常見題型。

時間複雜度： $O(N \log N)$

```cpp
// LIS O(N log N)。tail[k] = 長度 k+1 的遞增子序列「最小的結尾」
// 嚴格遞增：lower_bound；非嚴格（可相等）：upper_bound
int lis_length(const vector<int> &a) {
    vector<int> tail;
    for (int x : a) {
        auto it = lower_bound(tail.begin(), tail.end(), x); // [改] 非嚴格：upper_bound
        if (it == tail.end()) tail.push_back(x); else *it = x;
    }
    return tail.size(); // 注意 tail 的內容不一定是真正的 LIS
}
// 要輸出真正的 LIS：記錄每個元素接在誰後面
vector<int> lis_sequence(const vector<int> &a) {
    int n = a.size();
    vector<int> tail_val, tail_idx, prv(n, -1);
    for (int i = 0; i < n; i++) {
        int k = lower_bound(tail_val.begin(), tail_val.end(), a[i]) - tail_val.begin();
        if (k == (int)tail_val.size()) tail_val.push_back(a[i]), tail_idx.push_back(i);
        else tail_val[k] = a[i], tail_idx[k] = i;
        prv[i] = k ? tail_idx[k - 1] : -1;
    }
    vector<int> res;
    for (int i = tail_idx.empty() ? -1 : tail_idx.back(); i != -1; i = prv[i]) res.push_back(a[i]);
    reverse(res.begin(), res.end());
    return res;
}
// LDS：把每個數取負再做 LIS（嚴格遞減 → 對 -a 用 lower_bound；非嚴格遞減 → upper_bound）
/* [題型]
- 俄羅斯娃娃/信封 (w,h 都要嚴格大)：依 w 遞增排、w 相同時 h 遞減排，再對 h 做嚴格 LIS
- 橋不交叉：依南岸排序，對北岸做 LIS
- 最少幾個「遞減序列」覆蓋全部（攔截飛彈）= 嚴格遞增 LIS 長度 (Dilworth)
- 先增後減 (Bitonic)：L[i] = 以 i 結尾的 LIS、R[i] = 以 i 開頭的 LDS，max(L[i]+R[i]-1)
  以 i 結尾的 LIS 長度 = 插入位置 k + 1
- 最少刪幾個變遞增 = n - LIS；最少改幾個變嚴格遞增：對 a[i]-i 做非嚴格 LIS */
```

9.5 LCS 轉 LIS

用法： B 的每個數換成它在 A 中的位置，求嚴格 LIS = LCS。適合 A, B 為排列時。

時間複雜度： $O(K \log K)$，K 為配對數

```cpp
// 兩序列 LCS，當「每個值在 A 中出現次數很少」(例如 A、B 都是排列) 時 O(K log K)
// 把 B 中每個數換成它在 A 中的位置（由大到小放），對新序列求嚴格 LIS = LCS 長度
int lcs_by_lis(const vector<int> &A, const vector<int> &B) {
    unordered_map<int, vector<int>> pos;
    for (int i = 0; i < (int)A.size(); i++) pos[A[i]].push_back(i);
    vector<int> seq;
    for (int x : B) {
        auto it = pos.find(x);
        if (it == pos.end()) continue;
        for (int k = (int)it->second.size() - 1; k >= 0; k--)
            seq.push_back(it->second[k]); // 反向避免同一個 B 元素配到兩次
    }
    vector<int> tail;
    for (int x : seq) {
        auto p = lower_bound(tail.begin(), tail.end(), x);
        if (p == tail.end()) tail.push_back(x); else *p = x;
    }
    return tail.size();
}
```

9.6 區間 DP

用法： 由短區間推長區間。合併石頭通用模板、最長迴文子序列、變迴文的最少編輯 (含輸出)。

時間複雜度： $O(N^3)$ / $O(N^2)$

```cpp
/* 區間 DP：dp[l][r] 由「更短的區間」算出來 → 依長度由短到長（或 l 由大到小、r 由小到大）
   通用模板（合併石頭、矩陣鏈乘、切木棍）：枚舉分割點 k
     dp[l][r] = min_{l<=k<r} dp[l][k] + dp[k+1][r] + cost(l, r)   O(N^3)，N <= 500 */
const int MAXN = 505;
const ll INF = 1e18;
int n;
ll a[MAXN], pre[MAXN], dp[MAXN][MAXN];
ll merge_stones() { // 相鄰兩堆合併，代價 = 兩堆總和，求最小總代價 (a 為 1-based)
    for (int i = 1; i <= n; i++) pre[i] = pre[i - 1] + a[i], dp[i][i] = 0;
    for (int len = 2; len <= n; len++)
        for (int l = 1, r = len; r <= n; l++, r++) {
            dp[l][r] = INF;
            for (int k = l; k < r; k++) dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r]);
            dp[l][r] += pre[r] - pre[l - 1]; // [改] cost(l, r)
        }
    return dp[1][n]; // 環狀：複製一份接在後面（長度 2n），答案 min dp[i][i+n-1]
}
// 最長迴文子序列 (LPS)，0-based
int lps(const string &s) {
    int m = s.size();
    vector<vector<int>> f(m, vector<int>(m, 0));
    for (int i = m - 1; i >= 0; i--) {
        f[i][i] = 1;
        for (int j = i + 1; j < m; j++)
            f[i][j] = s[i] == s[j] ? f[i + 1][j - 1] + 2 : max(f[i + 1][j], f[i][j - 1]);
    }
    return m ? f[0][m - 1] : 0;
}
// 變成迴文的最少編輯（插入/刪除/取代都算 1），並輸出字典序最小的結果字串
pair<int, string> palindrome_edit(const string &s) {
    int m = s.size();
    if (!m) return {0, ""};
    vector<vector<int>> f(m + 1, vector<int>(m + 1, 0));
    for (int i = m - 1; i >= 0; i--)
        for (int j = i + 1; j < m; j++)
            f[i][j] = s[i] == s[j] ? f[i + 1][j - 1] : min({f[i + 1][j], f[i][j - 1], f[i + 1][j - 1]}) + 1;
    string L, R; int i = 0, j = m - 1;
    while (i < j) {
        if (s[i] == s[j]) { L += s[i]; R += s[i]; i++; j--; continue; }
        int best = 256, ch = 0; // 1 取代，2 處理左，3 處理右；同分取字典序小的字元
        if (f[i + 1][j - 1] + 1 == f[i][j] && min(s[i], s[j]) < best) best = min(s[i], s[j]), ch = 1;
        if (f[i + 1][j] + 1 == f[i][j] && s[i] < best) best = s[i], ch = 2;
        if (f[i][j - 1] + 1 == f[i][j] && s[j] < best) best = s[j], ch = 3;
        L += (char)best; R += (char)best;
        if (ch == 1) i++, j--; else if (ch == 2) i++; else j--;
    }
    if (i == j) L += s[i];
    reverse(R.begin(), R.end());
    return {f[0][m - 1], L + R};
}
/* 迴文子序列「總數」（位置不同算不同）：
   s[i]==s[j]: f = f[i+1][j] + f[i][j-1] + 1；否則 f = f[i+1][j] + f[i][j-1] - f[i+1][j-1] */
```

9.7 加權區間排程 (工作有獎勵)

用法： 選互不衝突的工作使獎勵和最大，並回溯輸出選了哪些。依結束時間排序 + 二分搜前一個不衝突的工作。`TOUCH_OK` 決定「端點相接」算不算衝突。

時間複雜度： $O(N \log N)$

```cpp
/* 加權區間排程：n 個工作 {開始 s, 結束 e, 獎勵 r}，選出互不衝突的工作使獎勵和最大，並輸出選了哪些
   （沒有獎勵、只求最多幾個 → Greedy「最多不重疊區間」即可）
   作法：依結束時間排序。dp[i] = 只看前 i 個工作（排序後）的最大獎勵
     dp[i] = max( dp[i-1],              不選第 i 個
                  dp[p[i]] + r[i] )     選第 i 個，p[i] = 排序後「結束時間不衝突」的工作有幾個
   p[i] 用二分搜在排好序的結束時間 ed[] 中找。O(N log N) */
const int MAXN = 200005;
int n, id_[MAXN], p[MAXN];
ll s[MAXN], e[MAXN], r[MAXN], ed[MAXN], dp[MAXN];
bool TOUCH_OK = true; // [改] true : 端點相接不算衝突 (e == 下一個的 s 可以接著做)
                      //       false: 端點相接也算衝突 (必須 e < 下一個的 s)
ll weighted_interval(vector<int> &chosen) { // 輸入 s[1..n], e[1..n], r[1..n]；chosen 回傳原始編號 (1-based)
    for (int i = 1; i <= n; i++) id_[i] = i;
    sort(id_ + 1, id_ + n + 1, [](int a, int b) { return make_pair(e[a], s[a]) < make_pair(e[b], s[b]); }); // 依結束時間排序（同時結束再比開始）
    for (int i = 1; i <= n; i++) ed[i] = e[id_[i]];
    dp[0] = 0;
    for (int i = 1; i <= n; i++) {
        ll st = s[id_[i]];
        if (TOUCH_OK) p[i] = upper_bound(ed + 1, ed + i, st) - (ed + 1); // 結束 <= st 的個數
        else          p[i] = lower_bound(ed + 1, ed + i, st) - (ed + 1); // 結束 <  st 的個數
        dp[i] = max(dp[i - 1], dp[p[i]] + r[id_[i]]);
    }
    chosen.clear(); // 回溯：dp[i] == dp[i-1] 代表沒選第 i 個，否則選了它並跳到 p[i]
    for (int i = n; i > 0;) {
        if (dp[i] == dp[i - 1]) i--;
        else chosen.push_back(id_[i]), i = p[i];
    }
    sort(chosen.begin(), chosen.end()); // [改] 要依時間順序輸出：改成依 s 排序
    return dp[n];
}
int main() { // 輸入：n，接著 n 行 s e r；輸出最大獎勵與選的工作編號
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> s[i] >> e[i] >> r[i];
    vector<int> chosen;
    cout << weighted_interval(chosen) << '\n';
    for (int i = 0; i < (int)chosen.size(); i++) cout << chosen[i] << " \n"[i + 1 == (int)chosen.size()];
    return 0;
}
/* 範例：(1,3,5) (2,4,6) (3,5,5) (4,6,4)
   TOUCH_OK = true : 10（選 1,3 或 2,4）   TOUCH_OK = false: 9（選 1,4）
   [變形]
   - 時間是「第幾天」且 [s, e] 兩天都要佔用（整數閉區間）→ TOUCH_OK = false
   - 求「最多幾個工作」→ r 全設 1（也可直接用 Greedy）
   - 時間很大（1e9）也沒關係，只用到排序和二分搜，不用開時間大小的陣列 */
```

9.8 Kadane 最大子陣列和

用法： 全為負數時回傳最大的負數。附二維、環狀等變形。

時間複雜度： $O(N)$

```cpp
// 最大連續子陣列和 O(N)。全部是負數時回傳最大的那個負數
ll kadane(const vector<ll> &a) {
    ll best = LLONG_MIN, cur = 0;
    for (ll x : a) {
```

```
        cur = max(x, cur + x); // 接在前面 vs 從自己重新開始
        best = max(best, cur);
    }
    return best; // 允許空陣列（答案至少 0）: best 初始 0
}
/* [變形]
- 最大子矩形和（N,M <= 300）: 枚舉上下列 r1..r2，把每行壓成一個數（行前綴和），對它做 kadane，O(N^2 M)
- 環狀陣列: max(一般 kadane, 總和 - 最小子陣列和)；全負時只取一般 kadane
- 長度至少 K: 前綴和 pre，ans = max(pre[i] - min(pre[0..i-K]))
- 最大子陣列乘積: 同時維護 cur_max、cur_min（負負得正）*/
```

9.9 跳格子 (路徑還原)

用法: 一次最多跳 K 格，最少步數並輸出路徑。
時間複雜度: $O(NK)$

```
/* PUPC 2025 River Crossing: v[i]=1 可踩、0 不可踩，一次最多跳 K 格，
   求從 0 到 n-1 最少跳幾次並輸出路徑（同步數時選 index 較小的前一格）。O(N*K)
   */
const int MAXN = 200005, INF = 0x3f3f3f3f;
int n, K = 3, v[MAXN], f[MAXN], pre_[MAXN];  // [改] K
void solve() {
    fill(f, f + n, INF); fill(pre_, pre_ + n, -1);
    f[0] = 0;
    for (int i = 1; i < n; i++) {
        if (!v[i]) continue;
        for (int j = 1; j <= K && i - j >= 0; j++)
            if (v[i - j] && f[i - j] != INF && f[i - j] + 1 <= f[i]) // <= : 同步數時換成更小的 index
                f[i] = f[i - j] + 1, pre_[i] = i - j;
    }
    if (f[n - 1] == INF) { cout << -1 << '\n'; return; }
    vector<int> path;
    for (int i = n - 1; i != -1; i = pre_[i]) path.push_back(i);
    reverse(path.begin(), path.end());
    cout << f[n - 1] << '\n';
    for (int i = 0; i < (int)path.size(); i++) cout << path[i] << " \n"[i + 1 == (int)path.size()];
}
// 若 K 很大: f[i] = min(f[i-K..i-1]) + 1 → 用單調隊列，O(N)
// 不用輸出路徑時: 用 Greedy「間隔跳躍」O(N)
```

10 Greedy

Greedy 不確定對不對時：用「交換論證」(相鄰兩個交換會不會更好) 推出排序方式；或寫暴力對拍。

10.1 區間覆蓋 (灑水器)

用法: 用最少區間蓋滿 $[0, L]$：依左端排序，每次選能延伸最遠的。
時間複雜度: $O(N \log N)$

```
/* 區間覆蓋: 用最少的區間蓋滿 [0, L]。灑水器版（UVa 10382）: 草地寬 W，
   灑水器在 pos、半徑 r，能蓋到的範圍是 pos ± sqrt(r^2 - (W/2)^2)
   作法: 依左端排序，每次在「左端 <= 目前已蓋到的位置」的區間中選右端最遠的 */
const double eps = 1e-9;
int min_cover(vector<pair<double, double>> seg, double L) { // seg = {左, 右}，無解回傳 -1
    sort(seg.begin(), seg.end());
    int cnt = 0, i = 0, n = seg.size();
    double reach = 0;                    // [改] 起點
    while (reach < L - eps) {
        double best = reach;
        while (i < n && seg[i].first <= reach + eps) best = max(best, seg[i].second), i++;
        if (best <= reach + eps) return -1; // 接不上
        reach = best; cnt++;             // 要輸出選了哪些: 記錄 best 是哪個區間
    }
    return cnt;
}
// 灑水器轉區間: if (r * 2 <= W) 跳過；half = sqrt(r*r - W*W/4.0); seg.push_back({pos - half, pos + half});
// 整數點版本（要蓋住點 1..L，[1,3] 與 [4,5] 算接起來）: 條件改成 seg[i].first <= reach + 1
```

10.2 區間選點

用法: 最少幾個點讓每個區間都含點：依右端排序。
時間複雜度: $O(N \log N)$

```
// 區間選點: 最少選幾個點，使每個區間 [l, r] 內至少有一個點。依「右端」排序，點盡量放右端
int min_points(vector<pair<int, int>> seg) { // seg = {l, r}
    sort(seg.begin(), seg.end(), [](auto &a, auto &b) { return a.second < b.second; });
    int cnt = 0; ll last = LLONG_MIN;
    for (auto [l, r] : seg)
        if (l > last) cnt++, last = r; // 這個區間還沒被蓋到 → 在它的右端放一個點
    return cnt;
}
// 答案也等於「最多能選幾個互不相交的區間」(同一個 greedy)
```

10.3 最多不重疊區間

用法: 依結束時間排序，能選就選。最少刪除數 $= N -$ 答案。
時間複雜度: $O(N \log N)$

```
// 活動選擇: 找到最多選幾個互不重疊的區間（或 = n - 最少刪除數）。依「結束時間」排序，能選就選
int max_non_overlap(vector<pair<int, int>> seg) { // {start, end}，[s, e) 半開區間
    sort(seg.begin(), seg.end(), [](auto &a, auto &b) { return a.second < b.second; });
    int cnt = 0; ll cur_end = LLONG_MIN; // 一開始什麼都沒選；第一個（最早結束的）會在迴圈裡被選到
    for (auto [s, e] : seg) // 寫法已保證 s == e（長度為 0）時也會成功選中
        if (s >= cur_end) cnt++, cur_end = e; // [改] 閉區間（端點碰到也算重疊）: s > cur_end
    return cnt; // cnt 直接是答案
}
// 另一種寫法（CSES Movie Festival）: cnt = 1, cur_end = 第一個的 end，迴圈「從第二個開始」
// 兩種起始值不能混用: cnt = 1 配 LLONG_MIN 會把第一個算兩次
// 有權重（每個活動有獎勵）→ 不能 Greedy，看 DP 的「加權區間排程」(WeightedInterval.cpp)
```

10.4 最少會議室

用法: 依開始時間排序，heap 存使用中房間的結束時間。
時間複雜度: $O(N \log N)$

```
// 最少會議室: 依開始時間排序，heap 存「每間使用中房間的結束時間」
int min_rooms(vector<pair<int, int>> seg) { // {start, end}
    sort(seg.begin(), seg.end());
    priority_queue<int, vector<int>, greater<int>> pq;
    for (auto [s, e] : seg) {
        if (!pq.empty() && pq.top() <= s) pq.pop(); // [改] 閉區間（結束當下不能接）: <
        pq.push(e);
    }
    return pq.size();
}
// 要輸出每個會議分到哪間: heap 改存 {結束時間, 房號}，pop 出來的房號給目前的會議
// 答案也等於「同一時間最多重疊幾個區間」→ 也可以用 SweepLine
```

10.5 掃描線：最大重疊數

用法: 開始 $+1$、結束 -1。**同時間的處理順序取決於開/閉區間**，見註解。
時間複雜度: $O(N \log N)$

```
// 掃描線: 同一時刻最多重疊幾個區間。開始 +1、結束 -1，依時間排序
int max_overlap(const vector<pair<int, int>> &seg) {
    vector<pair<int, int>> ev;
    for (auto [s, e] : seg) ev.push_back({s, +1}), ev.push_back({e, -1});
    sort(ev.begin(), ev.end()); // 同時間時 -1 排在 +1 前面
    int cur = 0, best = 0;
    for (auto [t, d] : ev) cur += d, best = max(best, cur);
    return best;
}
/* 同一時間點的順序很重要（和題目定義有關，看清楚！）:
   - 半開區間 [s, e)：e 時刻已離開 → 先 -1 再 +1（上面的預設）
   - 閉區間 [s, e]  ：e 時刻還在  → 先 +1 再 -1 → 把結束事件改成 {e + 1, -1}（整數時間）
   聯集總長度: 掃描時 cur > 0 的區段長度加總
   二維（矩形面積聯集）: x 方向掃描線 + 線段樹維護 y 方向被覆蓋的長度 */
```

10.6 最小化最大延遲

用法: 依期限由早到晚做。附 SJF、加權完成時間、交換論證。
時間複雜度: $O(N \log N)$

```
// 最小化最大延遲: 每個工作 {耗時, 期限}，一台機器依序做。依「期限」由早到晚做最好
ll min_max_lateness(vector<pair<ll, ll>> jobs) { // {time, deadline}
    sort(jobs.begin(), jobs.end(), [](auto &a, auto &b) { return a.second < b.second; });
    ll t = 0, ans = 0;
    for (auto [len, dl] : jobs) t += len, ans = max(ans, t - dl);
    return ans; // 0 代表全部準時
}
// 最小化「總完成時間」(或平均等待時間): 依耗時由短到長 (SJF)
// 加權版 Σ w_i * C_i: 依 time / weight 由小到大（比較時用交叉相乘避免浮點）
// 交換論證: 相鄰兩工作 i, j 交換前後的代價比較 → 得到排序的比較函式（自己寫 cmp 的萬用方法）
```

10.7 有期限的單位工作排程

用法: 最小化逾期懲罰。期限 $\le 10^6$ 用 DSU 找空位；期限很大用依期限排序 + min-heap。
時間複雜度: $O(N \log N)$

```
// 每個工作耗時 1、有期限 d 與懲罰 p（或利潤）。逾期要付懲罰，求最小總懲罰（= 最大化準時的利潤）
// 方法一: 懲罰大的先排，放在「期限內最晚的空閒時間」，用 DSU 找空位。近乎 O(N)，期限 <= 1e6
ll min_penalty_dsu(vector<pair<int, int>> jobs) { // {deadline, penalty}
    sort(jobs.begin(), jobs.end(), [](auto &a, auto &b) { return a.second > b.second; });
    int maxd = 0;
    for (auto [d, p] : jobs) maxd = max(maxd, d);
    vector<int> f(maxd + 1);
    iota(f.begin(), f.end(), 0);
    function<int(int)> find = [&](int x) { return f[x] == x ? x : f[x] = find(f[x]); };
    ll pen = 0;
```

```
    for (auto [d, p] : jobs) {
        int slot = find(min(d, maxd));
        if (slot > 0) f[slot] = slot - 1; // 佔用 slot，之後找到 slot 會跳到更早
        else pen += p;                    // 沒有空位 → 逾期
    }
    return pen;
}
// 方法二：期限很大 (1e9) 時，依期限排序 + min-heap 存已選工作的懲罰。O(N log N)
ll min_penalty_heap(vector<pair<int, int>> jobs) {
    sort(jobs.begin(), jobs.end());
    priority_queue<int, vector<int>, greater<int>> pq;
    ll total = 0, kept = 0;
    for (auto [d, p] : jobs) {
        total += p; pq.push(p); kept += p;
        if ((int)pq.size() > d) kept -= pq.top(), pq.pop(); // 排太多，丟掉懲罰最小的
    }
    return total - kept;
}
// 耗時不是 1（工作 {耗時 t, 期限 d}），最多完成幾個：依期限排序，累計時間超過 d 就丟掉耗時最長的 (max-heap)
```

10.8 多機排程 (LPT 近似)

用法： 耗時大的先做，交給最早空出來的機器。**只是近似解**。
時間複雜度： $O(N \log N)$

```
// m 台機器、n 個工作，最小化最晚完工時間 (NP-hard)。LPT 近似：耗時大的先做，交給最早空出來的機器
ll lpt_schedule(vector<ll> jobs, int m) {
    sort(jobs.rbegin(), jobs.rend());
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq; // {完工時間, 機器編號}
    for (int i = 0; i < m; i++) pq.push({0, i});
    for (ll t : jobs) {
        auto [done, id] = pq.top(); pq.pop();
        pq.push({done + t, id});
    }
    ll ans = 0;
    while (!pq.empty()) ans = max(ans, pq.top().first), pq.pop();
    return ans;
}
// 注意：這只是近似解！題目要精確最佳解時，可以「對答案二分搜 + 檢查能不能塞進 m 台」
```

10.9 跳躍遊戲 (最少跳幾次)

用法： a[i] 為從 i 最遠能跳的距離。
時間複雜度： $O(N)$

```
// Jump Game II: 站在 i 最遠可以跳到 i + a[i]，從 0 到 n-1 最少跳幾次 (到不了回傳 -1)。O(N)
// 想法：BFS 一層一層擴展，[目前這一跳能到的範圍] 內找下一跳最遠能到哪
int min_jumps(const vector<int> &a) {
    int n = a.size(), jumps = 0, cur_end = 0, farthest = 0;
    if (n <= 1) return 0;
    for (int i = 0; i < n - 1; i++) {
        if (i > farthest) return -1;          // 根本走不到 i
        farthest = max(farthest, i + a[i]);
        if (i == cur_end) {                   // 這一跳的範圍用完，必須再跳
            if (farthest <= i) return -1;
            jumps++; cur_end = farthest;
            if (cur_end >= n - 1) break;
        }
    }
    return cur_end >= n - 1 ? jumps : -1;
}
```

10.10 Josephus 約瑟夫問題

用法： 最後存活者。
時間複雜度： $O(N)$；k 小時 $O(k \log N)$

```
// n 個人 (編號 0..n-1) 圍圈，每數到第 k 個就淘汰，回傳最後存活者編號 (0-based)
int josephus(int n, int k) { // O(n)：J(1)=0, J(i) = (J(i-1) + k) % i
    int w = 0;
    for (int i = 1; i <= n; i++) w = (w + k) % i;
    return w;
}
ll josephus_fast(ll n, ll k) { // O(k log n)，n 很大 (1e18) 但 k 小時用
    if (n == 1) return 0;
    if (k == 1) return n - 1;
    if (k > n) return (josephus_fast(n - 1, k) + k) % n;
    ll res = josephus_fast(n - n / k, k) - n % k;
    if (res < 0) res += n;
    else res += res / (k - 1);
    return res;
}
/* 變形 (UVa 151 Power Crisis)：一定先淘汰 1 號，再每 k 個淘汰一個，求讓 13 號最後存活的最小 k
   → 1 號先淘汰後剩 n-1 人，從 2 號開始重新編號 0..n-2，13 號變成 11
   for k = 1..: if (josephus(n - 1, k) == 11) 答案為 k
   要知道「第 m 個被淘汰的人」：用 ordered_set / BIT 第 k 小模擬，O(n log n) */
```

10.11 Huffman 合併與編碼

用法： 每次合併最小的兩堆；陣列存樹產生編碼。
時間複雜度： $O(N \log N)$

```
// 合併 n 堆，每次合併代價 = 兩堆重量和，求最小總代價：每次挑最小的兩堆 (min-heap)
ll huffman_cost(const vector<ll> &w) {
    priority_queue<ll, vector<ll>, greater<ll>> pq(w.begin(), w.end());
    ll cost = 0;
    while (pq.size() > 1) {
        ll a = pq.top(); pq.pop();
        ll b = pq.top(); pq.pop();
        cost += a + b; pq.push(a + b);
    }
    return cost;
}
// Huffman 編碼：用陣列存樹 (lc, rc；-1 = 葉子)，葉子 i 對應字元 ch[i]
const int MAXN = 600;
int lc[MAXN], rc[MAXN], node_cnt;
char ch[MAXN];
map<char, string> code;
void get_code(int u, const string &s) {
    if (lc[u] == -1) { code[ch[u]] = s.empty() ? "0" : s; return; } // 只有一種字元時給 "0"
    get_code(lc[u], s + "0"); get_code(rc[u], s + "1");
}
int build_huffman(const string &text) { // 回傳根
    map<char, int> freq;
    for (char c : text) freq[c]++;
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq; // {頻率, 節點}
    node_cnt = 0; code.clear();
    for (auto [c, f] : freq) ch[node_cnt] = c, lc[node_cnt] = rc[node_cnt] = -1, pq.push({f, node_cnt++});
    while (pq.size() > 1) {
        auto [f1, a] = pq.top(); pq.pop();  // 最小的放左 (0)
        auto [f2, b] = pq.top(); pq.pop();  // [改] 題目規定左右/同頻率的順序時要改這裡
        lc[node_cnt] = a; rc[node_cnt] = b;
        pq.push({f1 + f2, node_cnt++});
    }
    int root = pq.top().second;
    get_code(root, "");
    return root; // 解碼：從 root 依 0/1 走 lc/rc，走到葉子輸出 ch 並回到 root
}
```

11 搜尋與技巧

11.1 Binary Search 二分搜

用法： 找第一個/最後一個可行的值、實數二分。附常見 check 寫法。
時間複雜度： $O(\log(\text{值域}) \times \text{check})$

```
/* 二分搜的本質：答案有「單調性」(x 可行 → 比 x 大的都可行，或反過來)
   題目問「最大值的最小值 / 最小值的最大值 / 最少需要多少才能...」→ 對答案二分搜 + check()
   寫 check(x) 時就當作「答案已知是 x」，問題通常會變成簡單的 greedy */
bool check(ll x) { return true; } // [改] x 可行嗎？

// 1. 找第一個 (最小) 可行的 x：check 長得像 F F F T T T
ll first_true(ll lo, ll hi) { // 答案在 [lo, hi]；全部不可行會回傳 hi+1
    hi++;
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (check(mid)) hi = mid; else lo = mid + 1;
    }
    return lo;
}
// 2. 找最後一個 (最大) 可行的 x：check 長得像 T T T F F F
ll last_true(ll lo, ll hi) { // 全部不可行會回傳 lo-1
    lo--;
    while (lo < hi) {
        ll mid = lo + (hi - lo + 1) / 2; // +1 才不會死迴圈
        if (check(mid)) lo = mid; else hi = mid - 1;
    }
    return lo;
}
// 3. 實數：固定跑 100 次 (比 while(r-l>eps) 安全)
double real_bs(double lo, double hi) {
    for (int it = 0; it < 100; it++) {
        double mid = (lo + hi) / 2;
        if (check(mid)) hi = mid; else lo = mid;
    }
    return lo;
}
/* 4. 陣列 (要先排序):
   lower_bound(a, a+n, x) - a: 第一個 >= x 的 index；upper_bound: 第一個 > x
   x 出現次數 = upper - lower；<= x 的個數 = upper_bound(...) - a
   [常見 check]
   - 切成 <= k 段，使每段和的最大值最小：check(x) = greedy 切，每段和 <= x，算段數 <= k
   - 放 k 個東西，使最近距離最大 (Aggressive cows)：check(x) = 從左邊開始，距離 >= x 就放
   - 第 k 小的「某種組合」(例：兩兩差、矩陣乘法表)：二分值 x，check = 「<= x 的有幾個」>= k
   - 平均值最大 (分數規劃)：二分 x，每個數減 x 後看能否 >= 0 */
// [例] 構造：給前綴和的正負號 (+,-,0)，求元素皆非 0 時「最大絕對值」的最小值
// check(M)：維護前綴和可能範圍 [L, R]，每步 L -= M, R += M 再依符號截斷，L > R 不可行
```

11.2 Ternary Search 三分搜

用法： 單峰/單谷函數極值。浮點數固定跑 200 次；整數剩 3 個以內暴力。
時間複雜度： $O(\log(\text{值域}))$

```
// 單峰/單谷函數求極值。以「求最小值」為例；求最大值把 > 改成 <、min 改 max
double f(double x) { return (x - 2) * (x - 2); } // [改] 目標函數
double ternary_double(double L, double R) { // 回傳極值所在的 x
    for (int it = 0; it < 200; it++) {
```

```
        double m1 = L + (R - L) / 3, m2 = R - (R - L) / 3;
        if (f(m1) > f(m2)) L = m1; else R = m2;
    }
    return L;
}
ll fi(ll x) { return (x - 2) * (x - 2); } // 整數版目標函數
ll ternary_int(ll L, ll R) { // 回傳最小的函數值
    while (R - L > 2) {
        ll m1 = L + (R - L) / 3, m2 = R - (R - L) / 3;
        if (fi(m1) > fi(m2)) L = m1; else R = m2;
    }
    ll best = fi(L);
    for (ll x = L + 1; x <= R; x++) best = min(best, fi(x)); // 剩 <= 3 個直接暴力
    return best;
}
// 注意：函數有「平台」(連續相等的值) 時三分搜可能失敗 → 整數可改二分搜 f(x) 與 f(x+1) 的大小關係
```

11.3 Two Pointer 雙指針

用法: 同向 (視窗) 與對向 (排序後)。
時間複雜度: $O(N)$

```
/* 雙指針 (同向): r 每次往右一格，l 在「條件被破壞」時往右縮。總共 O(N)
   能用的條件：區間變大條件只會更難滿足 (單調性)，例如「和 <= x 且元素非負」 */
const int MAXN = 200005;
int n, a[MAXN];
// 1. 和 <= x 的最長子陣列 (a 非負)
int longest_sum_at_most(ll x) {
    ll sum = 0; int best = 0;
    for (int l = 0, r = 0; r < n; r++) {
        sum += a[r];
        while (sum > x) sum -= a[l++]; // [改] 縮的條件
        best = max(best, r - l + 1);
    }
    return best;
}
// 2. 和 >= target 的最短子陣列 (a 非負)，找不到回傳 0
int shortest_sum_at_least(ll target) {
    ll sum = 0; int best = INT_MAX;
    for (int l = 0, r = 0; r < n; r++) {
        sum += a[r];
        while (l <= r && sum >= target) best = min(best, r - l + 1), sum -= a[l++];
    }
    return best == INT_MAX ? 0 : best;
}
// 3. 對向: 排序後找兩數和 = x (3 Sum: 固定第一個數，剩下做 2 Sum，O(N^2))
bool two_sum(ll x) { // a 已排序
    for (int l = 0, r = n - 1; l < r;) {
        ll s = a[l] + a[r];
        if (s == x) return true;
        s < x ? l++ : r--;
    }
    return false;
}
// 有負數時「和 = k 的子陣列」不能用雙指針 → 前綴和 + map
// 4. Floyd 判圈 (函數迭代 x → f(x) 找環): slow 走 1 步、fast 走 2 步，相遇代表有環
```

11.4 Sliding Window 滑動視窗

用法: 最長不重複子字串、最多 K 種值的子陣列數。
時間複雜度: $O(N)$ (用 map 時 $O(N \log N)$)

```
// 滑動視窗 = 同向雙指針 + 維護視窗內的統計 (計數陣列/map)
// 1. 最長「不含重複字元」的子字串
int longest_unique(const string &s) {
    int cnt[256] = {}, best = 0;
    for (int l = 0, r = 0; r < (int)s.size(); r++) {
        cnt[(unsigned char)s[r]]++;
        while (cnt[(unsigned char)s[r]] > 1) cnt[(unsigned char)s[l++]]--;
        best = max(best, r - l + 1);
    }
    return best;
}
// 2. 「最多 K 種不同值」的子陣列個數 (恰好 K 種 = atMost(K) - atMost(K-1))
ll at_most_k_distinct(const vector<int> &a, int k) {
    map<int, int> cnt; ll res = 0;
    for (int l = 0, r = 0; r < (int)a.size(); r++) {
        cnt[a[r]]++;
        while ((int)cnt.size() > k) { if (--cnt[a[l]] == 0) cnt.erase(a[l]); l++; }
        res += r - l + 1; // 以 r 結尾的合法子陣列有 r-l+1 個
    }
    return res;
}
// 3. 固定長度 len 的子陣列最小和: 先算前 len 個，之後每步 +a[i] -a[i-len]
// 視窗內最大/最小值 → 單調隊列 (MonotonicQueue)
// 包含 t 所有字元的最短子字串: need[] 計數，滿足時一直縮左邊並更新答案
```

11.5 暴力枚舉、回溯、折半枚舉

用法: 子集合/排列枚舉、排列 vs 組合、回溯剪枝 (N 皇后)、$N \le 40$ 的折半枚舉、選/不選遞迴 (分兩組)、有重複字母的相異排列。
時間複雜度: $O(2^N)$ / $O(2^{N/2}N)$

```
// 1. 枚舉所有子集合 (n <= 20)
//    for (int mask = 0; mask < (1 << n); mask++) for (i) if (mask >> i & 1) ... 選了第 i 個
// 2. 枚舉所有排列 (n <= 10): 先 sort，do { ... } while (next_permutation(a.begin(), a.end()));
// 排列 vs 組合 (先問: 順序重不重要？)
//   排列 N!: 每層 for i = 0..n-1 跳過 used[i]；used[i] = 1; dfs(); used[i] = 0; ← 一定要還原
//   組合 2^N: 只看「選了哪些」→ 下一個只能從 i+1 開始選，或用下面 5. 的選/不選
//   只標記不還原 → 第二個分支起會漏枚舉；組合題寫成排列 → 同一組被走 k! 次而 TLE
// 3. 回溯 + 剪枝 (例: N 皇后)
int n, cnt_;
bool col[20], d1[40], d2[40];
void queens(int r) {
    if (r == n) { cnt_++; return; }                    // [改] 找到一組解
    for (int c = 0; c < n; c++) {
        if (col[c] || d1[r + c] || d2[r - c + n]) continue; // 剪枝: 不合法就不往下
        col[c] = d1[r + c] = d2[r - c + n] = true;
        queens(r + 1);
        col[c] = d1[r + c] = d2[r - c + n] = false;    // 回溯: 還原
    }
}
// 剪枝技巧: 目前答案已經比 best 差就 return；先試「比較可能成功」的選項；記憶化相同狀態

// 4. 折半枚舉 (Meet in the middle): n <= 40，子集合和 = target 的個數，O(2^(n/2) * n)
vector<ll> half_sums(const vector<ll> &v) {
    vector<ll> s = {0};
    for (ll x : v) { int k = s.size(); for (int i = 0; i < k; i++) s.push_back(s[i] + x); }
    return s;
}
ll count_subset_sum(const vector<ll> &a, ll target) {
    int h = a.size() / 2;
    vector<ll> L = half_sums(vector<ll>(a.begin(), a.begin() + h));
    vector<ll> R = half_sums(vector<ll>(a.begin() + h, a.end()));
    sort(R.begin(), R.end());
    ll res = 0;
    for (ll x : L) res += upper_bound(R.begin(), R.end(), target - x) - lower_bound(R.begin(), R.end(), target - x);
    return res; // [改] 最接近 target: 對每個 x 在 R 中 lower_bound 找最近的
}

// 5. 選 / 不選 (分成兩組): 每層只決定第 idx 個放哪，O(2^n)。例: CSES 1623 兩組重量差最小
ll a[45];
ll split(int idx, ll L, ll R) {
    if (idx == n) return llabs(L - R);                 // [改] 全部分完才算答案
    return min(split(idx + 1, L + a[idx], R),          // 第 idx 個放左
               split(idx + 1, L, R + a[idx]));         // 第 idx 個放右
}   // 呼叫 split(0, 0, 0)；總和可到 2e10 用 ll；第 0 個固定放左可砍半
// 答案是兩組的差 |L - R| = |total - 2L|，不是 |L - total/2| (total 奇數時整數除法會錯)

// 6. 有重複字母的「相異」排列 (CSES 1622): 記「每個字母還剩幾個」，不記「哪個位置用過」
//    每層從 a 試到 z → 同一層同一個字母只走一次 → 自動去重，產生順序就是字典序 (不用 set)
int cnt[26]; string cur; vector<string> res;  // 先 for (char c : s) cnt[c - 'a']++; n = s.size()
void perm() {
    if ((int)cur.size() == n) { res.push_back(cur); return; }
    for (int i = 0; i < 26; i++) if (cnt[i]) {
        cnt[i]--; cur.push_back('a' + i);  // 共用的狀態「每一個」都要做…
        perm();
        cnt[i]++; cur.pop_back();          // …也都要還原 (只還原字串、沒還原 cnt → 變成可重複選)
    }
}   // 要先輸出總數 → 先存進 res，最後印 res.size() 再印內容
// 傳值 perm(cur + c) 也對 (回來不用 pop)，但每層複製字串；狀態大時用共用 + 還原
// 另解: sort(s) 後 do {...} while (next_permutation(s.begin(), s.end())) 也只產生相異排列
```

12 MISC

12.1 安全的 Hash

用法: unordered_map 被特殊測資卡成 $O(N)$ 時使用。
時間複雜度: 均攤 $O(1)$

```
// unordered_map 被惡意測資卡成 O(N) 時 (Codeforces 常見)，換成隨機化的 hash
uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}
const uint64_t RND = chrono::steady_clock::now().time_since_epoch().count();
auto safe_hash = [](uint64_t x) { return splitmix64(x + RND); };
unordered_map<ll, int, decltype(safe_hash)> mp(1 << 16, safe_hash);
unordered_set<ll, decltype(safe_hash)> st_(1 << 16, safe_hash);
// pair 當 key: 合成一個 ll，例如 (ll)a * 1000000007 + b，或直接用 map (O(log N) 通常也夠)
// 加速: mp.reserve(1 << 20); mp.max_load_factor(0.25);
```

12.2 對拍器 (測資產生器)

用法: 有一個解法、也有保證正確的暴力解時，用隨機小測資找出反例。
產生器: 陣列、排列、字串、區間、各種形狀的樹 (隨機/鏈/星/二元/毛毛蟲)、一般圖、連通圖、DAG、完全圖、網格。

```
// ===== gen.cpp: 隨機測資產生器。N 與數值都要調小 (N <= 10、值 <= 100)，WA 時才看得懂 =====
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll rnd(ll L, ll R) { return uniform_int_distribution<ll>(L, R)(rng); } // [L, R] 的隨機整數

// ---------- 1. 陣列與字串 ----------
void gen_array(int n, ll L, ll R) { for (int i = 0; i < n; i++) cout << rnd(L, R) << " \n"[i == n - 1]; }
```

```cpp
void gen_permutation(int n) { // 1-n 不重複
    vector<int> p(n); iota(p.begin(), p.end(), 1); shuffle(p.begin(), p.end(), rng);
    for (int i = 0; i < n; i++) cout << p[i] << " \n"[i == n - 1];
}
void gen_few_distinct(int n, ll L, ll R, int k) { // 只有 k 種不同的值（測重複值）
    vector<ll> pool(k); for (ll &x : pool) x = rnd(L, R);
    for (int i = 0; i < n; i++) cout << pool[rnd(0, k - 1)] << " \n"[i == n - 1];
}
void gen_sorted(int n, ll L, ll R, bool rev) { // 已排序 / 完全反序
    vector<ll> a(n); for (ll &x : a) x = rnd(L, R);
    sort(a.begin(), a.end()); if (rev) reverse(a.begin(), a.end());
    for (int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];
}
void gen_string(int n, string cs = "abc") { for (int i = 0; i < n; i++) cout << cs[rnd(0, cs.size() - 1)]; cout << '\n'; }
void gen_intervals(int n, ll mx) { // n 個區間 [l, r]
    for (int i = 0; i < n; i++) { ll l = rnd(1, mx), r = rnd(1, mx); if (l > r) swap(l, r); cout << l << ' ' << r << '\n'; }
}

// ---------- 2. 樹（n 個點、n-1 條邊）----------
// 共用：打亂點的編號與邊的順序，避免「編號剛好有順序」讓錯的 code 也 AC
void print_tree(int n, vector<pair<int, int>> e, bool weighted = false, ll maxw = 10) {
    vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1, p.end(), rng);
    shuffle(e.begin(), e.end(), rng);
    for (auto [u, v] : e) {
        u = p[u]; v = p[v]; if (rnd(0, 1)) swap(u, v);
        cout << u << ' ' << v; if (weighted) cout << ' ' << rnd(1, maxw);
        cout << '\n';
    }
}
void gen_random_tree(int n, bool w = false) { vector<pair<int, int>> e; for (int i = 2; i <= n; i++) e.push_back({(int)rnd(1, i - 1), i}); print_tree(n, e, w); }
void gen_chain(int n, bool w = false) { vector<pair<int, int>> e; for (int i = 2; i <= n; i++) e.push_back({i - 1, i}); print_tree(n, e, w); } // 一條線（卡遞迴深度）
void gen_star(int n, bool w = false) { vector<pair<int, int>> e; for (int i = 2; i <= n; i++) e.push_back({1, i}); print_tree(n, e, w); }      // 一個中心（卡 O(度數 ^2)）
void gen_binary_tree(int n, bool w = false) { vector<pair<int, int>> e; for (int i = 2; i <= n; i++) e.push_back({i / 2, i}); print_tree(n, e, w); } // 完全二元樹
void gen_caterpillar(int n, bool w = false) { // 毛毛蟲：一條主幹，其他點都直接掛在主幹上
    vector<pair<int, int>> e; int len = rnd(1, n);
    for (int i = 2; i <= len; i++) e.push_back({i - 1, i});
    for (int i = len + 1; i <= n; i++) e.push_back({(int)rnd(1, len), i});
    print_tree(n, e, w);
}

// ---------- 3. 圖（n 個點、m 條邊）----------
void gen_graph(int n, int m, bool w = false, bool directed = false) { // 無自環、無重邊。m <= n(n-1)/2
    set<pair<int, int>> e;
    while ((int)e.size() < m) {
        int u = rnd(1, n), v = rnd(1, n);
        if (u == v) continue;
        if (!directed && u > v) swap(u, v);
        e.insert({u, v});
    }
    vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(), ev.end(), rng);
    for (auto [u, v] : ev) { cout << u << ' ' << v; if (w) cout << ' ' << rnd(1, 10); cout << '\n'; }
}
void gen_connected_graph(int n, int m, bool w = false) { // 保證連通：先產生一棵隨機樹，再補 m-(n-1) 條邊
    set<pair<int, int>> e;
    for (int i = 2; i <= n; i++) e.insert({(int)rnd(1, i - 1), i});
    while ((int)e.size() < m) { int u = rnd(1, n), v = rnd(1, n); if (u != v) e.insert({min(u, v), max(u, v)}); }
    vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1, p.end(), rng);
    vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(), ev.end(), rng);
    for (auto [u, v] : ev) { cout << p[u] << ' ' << p[v]; if (w) cout << ' ' << rnd(1, 10); cout << '\n'; }
}
void gen_dag(int n, int m, bool w = false) { // 有向無環：只產生「小編號 → 大編號」的邊，再打亂編號
    set<pair<int, int>> e;
    while ((int)e.size() < m) { int u = rnd(1, n - 1); e.insert({u, (int)rnd(u + 1, n)}); }
    vector<int> p(n + 1); iota(p.begin(), p.end(), 0); shuffle(p.begin() + 1, p.end(), rng);
    vector<pair<int, int>> ev(e.begin(), e.end()); shuffle(ev.begin(), ev.end(), rng);
    for (auto [u, v] : ev) { cout << p[u] << ' ' << p[v]; if (w) cout << ' ' << rnd(1, 10); cout << '\n'; }
}
void gen_complete_graph(int n, bool w = false) { gen_graph(n, n * (n - 1) / 2, w); } // 完全圖（卡 O(E^2)）
void gen_grid(int r, int c, int wall_pct = 20) { // 網格地圖：'#' 牆、'.' 空地
    cout << r << ' ' << c << '\n';
    for (int i = 0; i < r; i++) { for (int j = 0; j < c; j++) cout << (rnd(1, 100) <= wall_pct ? '#' : '.'); cout << '\n'; }
}

int main() { // [改] 依題目的輸入格式組合上面的函式
    int T = 1;            // 多筆測資時：T = rnd(1, 3); cout << T << '\n';
    while (T--) {
        int n = rnd(2, 8), m = rnd(n - 1, n * (n - 1) / 2);
        cout << n << ' ' << m << '\n';
        gen_connected_graph(n, m, true);
    }
}
/* ===== 使用步驟 =====
1. 同一個資料夾放：sol.cpp（你的解）、brute.cpp（暴力但保證對）、gen.cpp、check.sh 或 check.bat
2. 改 gen.cpp 的 main，產生符合題目格式的「小」測資
3. 執行 check，出現 WA 時：in.txt = 讓你錯的測資，ans.txt = 正確答案，out.txt = 你的答案

===== check.sh (Linux / Git Bash)，第一次先 chmod +x check.sh =====
g++ -O2 gen.cpp -o gen && g++ -O2 sol.cpp -o sol && g++ -O2 brute.cpp -o brute
for ((i = 1; ; i++)); do
    ./gen > in.txt; ./sol < in.txt > out.txt; ./brute < in.txt > ans.txt
    if ! diff -w out.txt ans.txt > /dev/null; then echo "WA on test $i"; cat in.txt; break; fi
    echo "Test $i: AC"
done

===== check.bat (Windows)，先手動編譯成 gen.exe / sol.exe / brute.exe =====
:loop
gen > in.txt & sol < in.txt > out.txt & brute < in.txt > ans.txt
fc /w out.txt ans.txt > nul || (echo WA & type in.txt & exit /b)
goto loop
*/
```